

02__estimacion__recursos

September 24, 2026

1 Estimación de recursos de cómputo, almacenamiento y transferencia

A continuación, vamos a estimar el costo de procesar un lote fijo de 89 muestras de la base de datos de ENA (PRJEB11755). Para hacer esto, primero hacemos una medición real de la descarga completa de una muestra aleatoria. Luego, usando los metadatos del tamaño real de las muestras de ENA, buscamos estimar cuántos recursos de cómputo tendrían que usarse para la descarga y el pre-procesado de todos los datos.

Se entrega la medición en núcleo-horas de cómputo, y se estima la velocidad usando la velocidad teórica de los dispositivos del HPC, comparándolo con la velocidad obtenida en una red doméstica.

1.1 Medición real y en limpio de una corrida individual

Reproduce, con la misma configuración (`THREADS=12`, `FRACCION_SECCION=0.10`, `N_SECCIONES=2`) el procesamiento completo de una muestra nunca antes descargada ni procesada en este repositorio, midiendo cada paso con herramientas del sistema.

Solo se seleccionan corridas `PAIRED` (ver `seleccionar_runs()` en `seleccion_muestras.py`) – el pipeline de descarga y submuestreo asume siempre dos archivos `FASTQ` (`R1/R2`) y no reconoce `library_layout=SINGLE`.

Por cada muestra descargada y filtrada por QC, se saca un pool aleatorio (`seqtk sample -s{SEED}`) del 20% de los pares de lecturas, y se parte en dos secciones consecutivas de 10% cada una, sin solapamiento entre sí (ver sección 1.3). Cada sección se trata como un elemento distinto aguas abajo (sufijos `_1/_2`): ensamblado, predicción de genes y detección de ARG corren una vez por sección, no una sola vez por muestra.

La detección de ARG corre contra dos bases en paralelo: `CARD` (`rgi`) y `NCBI AMRFinderPlus` (`amrfinder`), sin descartar ninguna – se cruzan por gen y por familia/clase de droga para ver dónde coinciden y dónde cada una encuentra algo que la otra no. Además de la detección, cada ARG queda ubicado dentro de su contig: coordenadas, cadena y posición relativa (0-1).

- Tiempo y RAM pico: `/usr/bin/time -v` alrededor de cada comando externo (`fastp`, el submuestreo (`seqtk`), `megahit`, `prodigal`, `rgi`, `amrfinder`) – reporta el tiempo de pared real y el `Maximum resident set size` real del proceso, no un estimado.
- Descarga: `aria2c` (multi-conexión), medido con `/usr/bin/time -v` igual que el resto de los pasos externos – ya no es un caso especial de `urllib` puro cronometrado a mano. Se calcula la velocidad real lograda (`bytes / segundos`) en vez de asumir una.

- Disco: tamaño real en bytes de los archivos que deja cada paso (`os.path.getsize` / recorrido de directorio).

```
[1]: import pandas as pd
import os
from pathlib import Path
import re
import subprocess
import time
import hashlib
import urllib.error
import urllib.request
import json
import shutil
from Bio import SeqIO
import sys
```

```
[18]: MUESTREO_XLSX = "muestreo_PAired_89.xlsx"

muestreo_89 = pd.read_excel(MUESTREO_XLSX, sheet_name="Muestra_PAired_89")
resumen_muestreo = pd.read_excel(MUESTREO_XLSX, sheet_name="Resumen")
RUNS = list(muestreo_89["run_accession"])

UNIVERSO_PAired_TOTAL = int(
    resumen_muestreo.loc[resumen_muestreo["grupo"] == "TOTAL",
    ↪ "PAired_disponibles"].iloc[0]
)

THREADS          = 12
FRACCION_SECCION = 0.10
N_SECCIONES      = 2
SEED              = 100

print(f"{len(RUNS)} muestras configuradas (subconjunto fijo de {MUESTREO_XLSX},
↪
    f"de un universo de {UNIVERSO_PAired_TOTAL} corridas PAired profundas en
↪ PRJEB11755)")
```

89 muestras configuradas (subconjunto fijo de `muestreo_PAired_89.xlsx`, de un universo de 295 corridas PAired profundas en PRJEB11755)

```
[3]: def procesada(run):
    carpetas = ["raw", "work", "results"]
    return any(Path(f"{d}/{run}").exists() for d in carpetas)

MUESTRA_REF = next(r for r in RUNS if not procesada(r))
```

```

RAW_DIR, WORK_DIR, OUT_DIR = Path(f"raw/{MUESTRA_REF}"), Path(f"work/
↳{MUESTRA_REF}"), Path(f"results/{MUESTRA_REF}")
for d in (RAW_DIR, WORK_DIR, OUT_DIR):
    d.mkdir(parents=True, exist_ok=True)

print(f"Muestra de referencia para esta corrida: {MUESTRA_REF}")

```

Muestra de referencia para esta corrida: ERR1135179

```

[11]: def _segundos(txt_gnu_time):
    partes = [float(p) for p in txt_gnu_time.split(":")]
    while len(partes) < 3:
        partes.insert(0, 0.0)
    h, m, s = partes
    return h * 3600 + m * 60 + s

def _parsear_time_v(salida):
    m_ram = re.search(r"Maximum resident set size \((kbytes\):\s*(\d+)", salida)
    m_t = re.search(r"Elapsed \((wall clock\) time.*?:\s*(\d[:.]+)", salida)
    m_cpu = re.search(r"Percent of CPU this job got:\s*(\d+)%", salida)
    ram_mb = int(m_ram.group(1)) / 1024 if m_ram else None
    tiempo_s = _segundos(m_t.group(1)) if m_t else None
    cpu_pct = int(m_cpu.group(1)) if m_cpu else None
    nucleo_s = tiempo_s * cpu_pct / 100 if (tiempo_s is not None and cpu_pct is
↳not None) else None
    return tiempo_s, ram_mb, nucleo_s

def sh_medido(cmd):
    t0 = time.time()
    proc = subprocess.Popen(f"/usr/bin/time -v {cmd}", shell=True, text=True,
↳bufsize=1,
                                stdout=subprocess.PIPE, stderr=subprocess.STDOUT)

    lineas = []
    for line in proc.stdout:
        lineas.append(line)
        el = time.time() - t0
        print(f"[{int(el // 60):>2}m{int(el % 60):02d}s] {line}", end="",
↳flush=True)
    proc.wait()
    salida = "".join(lineas)
    if proc.returncode != 0:
        raise RuntimeError(f"Fallo (codigo {proc.returncode}): {cmd}\n{salida}")

    tiempo_s, ram_mb, nucleo_s = _parsear_time_v(salida)

```

```

    print(f"Completado en {int((time.time() - t0) // 60)}m{int((time.time() - t0) % 60):02d}s")
    return tiempo_s, ram_mb, nucleo_s, salida

def tamano(ruta):
    ruta = Path(ruta)
    if not ruta.exists():
        return 0
    if ruta.is_file():
        return ruta.stat().st_size
    return sum(f.stat().st_size for f in ruta.rglob("*") if f.is_file())

mediciones = []

```

1.1.1 Descarga (ENA)

Se realiza la conexión con la API de ENA para descargar las muestras que se desean procesar. Para la muestra de referencia (MUESTRA_REF), la descarga se hace **dos veces seguidas** – primero con `urllib` (una sola conexión) y, tras borrar esa copia, con `aria2c` (multi-conexión) – para medir y comparar el tiempo real de cada método contra el mismo archivo y la misma red, en vez de asumir cuál es más rápido. Solo se cuantifica el tiempo de cada descarga en sí; el borrado intermedio no se mide. La copia que queda en disco al final (la de `aria2c`) es la que alimenta el resto del pipeline – no se vuelve a descargar después del experimento.

```

[12]: def _human(n):
    for u in ["B", "KB", "MB", "GB"]:
        if n < 1024:
            return f"{n:.1f}-{u}"
        n /= 1024
    return f"{n:.1f}TB"

def md5sum(path, chunk=1 << 20):
    h = hashlib.md5()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(chunk), b''):
            h.update(block)
    return h.hexdigest()

def descargar_con_aria2c(pares_url_nombre, dest_dir, threads,
    bytes_totales_esperados=None, intervalo_reporte_s=0.2):
    input_file = dest_dir / "aria2_input.txt"
    with open(input_file, "w") as f:
        for url, nombre in pares_url_nombre:

```

```

        f.write(f"{url}\n out={nombre}\n")

    conexiones = min(threads, 16)
    archivos = [Path(dest_dir) / nombre for _, nombre in pares_url_nombre]
    cmd = (
        f"/usr/bin/time -v aria2c -i {input_file} -d {dest_dir} -x{conexiones}\n
↪-s{conexiones} -j2 -c "
        f"--max-tries=0 --retry-wait=5 --timeout=120 --allow-overwrite=true\n
↪--quiet=true"
    )

    t0 = time.time()
    proc = subprocess.Popen(cmd, shell=True, text=True, bufsize=1,
                             stdout=subprocess.PIPE, stderr=subprocess.STDOUT)

    last_print = 0.0
    while proc.poll() is None:
        now = time.time()
        if now - last_print >= intervalo_reporte_s:
            last_print = now
            done = sum(tamano(a) for a in archivos)
            el = now - t0
            spd = done / el if el > 0 else 0
            if bytes_totales_esperados:
                pct = done / bytes_totales_esperados * 100
                eta = (bytes_totales_esperados - done) / spd if spd > 0 else 0
                print(f"\r aria2c: {pct:5.1f}% {_human(done)}/
↪{_human(bytes_totales_esperados)} "
                    f"{_human(spd)}/s ETA {int(eta // 60)}m{int(eta % 60):
↪02d}s ", end="", flush=True)
            else:
                print(f"\r aria2c: {_human(done)} descargados {_human(spd)}/s\n
↪ ", end="", flush=True)
            time.sleep(0.05)

        salida = proc.stdout.read()
        print()
        if proc.returncode != 0:
            raise RuntimeError(f"Fallo (codigo {proc.returncode}): {cmd}\n{salida}")

    tiempo_s, ram_mb, nucleo_s = _parsear_time_v(salida)
    return tiempo_s, ram_mb, nucleo_s, salida

def descargar_con_urllib(pares_url_nombre, dest_dir, chunk=1 << 18,
↪max_retries=1000):
    t0_total = time.time()

```

```

for url, nombre in pares_url_nombre:
    out = Path(dest_dir) / nombre
    attempt = 0
    last_print = 0.0
    while True:
        existing = out.stat().st_size if out.exists() else 0
        headers = {"Range": f"bytes={existing}-"} if existing else {}
        try:
            with urllib.request.urlopen(urllib.request.Request(url,
↳headers=headers), timeout=120) as resp:
                if existing and getattr(resp, "status", 200) == 206:
                    mode = "ab"
                    total = existing + int(resp.headers.
↳get("Content-Length", 0))
                else:
                    existing, mode = 0, "wb"
                    total = int(resp.headers.get("Content-Length", 0))
                    done, t0 = existing, time.time()
                    with open(out, mode) as f:
                        while True:
                            block = resp.read(chunk)
                            if not block:
                                break
                            f.write(block)
                            done += len(block)
                            now = time.time()
                            if now - last_print >= 0.2 or (total and done >=
↳total):
                                last_print = now
                                el = now - t0
                                spd = (done - existing) / el if el > 0 else 0
                                pct = (done / total * 100) if total else 0
                                eta = (total - done) / spd if spd > 0 else 0
                                print(f"\r {out.name}: {pct:5.1f}%
↳{_human(done)}/{_human(total)} "
                                f"{_human(spd)}/s ETA {int(eta //
↳60)}m{int(eta % 60):02d}s ",
                                end="", flush=True)
                                if total and done < total:
                                    raise IOError(f"conexion cerrada antes de tiempo:
↳{_human(done)}/{_human(total)}")
                                print()
                                break
                            except urllib.error.HTTPError as e:
                                if e.code == 416:

```

```

        print(f"\n Rango invalido; reinicio {out.name} desde cero.
↪")
        out.unlink(missing_ok=True)
        continue
    raise
except Exception as e:
    attempt += 1
    if attempt > max_retries:
        raise
    print(f"\n Interrumpido ({e}); reintentando en 5s [{attempt}]..
↪.", flush=True)
    time.sleep(5)
return time.time() - t0_total

def reintentar(fn, intentos=5, espera_s=5, descripcion="operacion"):
    ultimo_error = None
    for intento in range(1, intentos + 1):
        try:
            return fn()
        except Exception as e:
            ultimo_error = e
            if intento < intentos:
                print(f" Fallo en {descripcion} ({e}); reintentando en
↪{espera_s}s [{intento}/{intentos}]...")
                time.sleep(espera_s)
            raise RuntimeError(f"'{descripcion}' fallo tras {intentos} intentos:
↪{ultimo_error}") from ultimo_error

def _consultar_ena_filereport(url, descripcion):
    def _get():
        df = pd.read_csv(url, sep="\t")
        if df.empty or "fastq_ftp" not in df.columns or pd.isna(df.loc[0,
↪"fastq_ftp"]):
            raise ValueError(f"respuesta de ENA sin fastq_ftp utilizable: {df.
↪to_dict('records')[0]}")
        return df
    return reintentar(_get, descripcion=descripcion)

```

```

[13]: ena_url = (
    "https://www.ebi.ac.uk/ena/portal/api/filereport"
    f"?
↪accession={MUESTRA_REF}&result=read_run&fields=fastq_ftp,fastq_md5,fastq_bytes&format=tsv"
)

```

```

ena = _consultar_ena_filereport(ena_url, descripcion=f"consulta ENA_
↳({MUESTRA_REF})")
ftp = ena.loc[0, "fastq_ftp"].split(";")
md5_esperado = ena.loc[0, "fastq_md5"].split(";")
bytes_totales_esperados = sum(int(b) for b in ena.loc[0, "fastq_bytes"].split(";
↳"))

r1, r2 = RAW_DIR / f"{MUESTRA_REF}_1.fastq.gz", RAW_DIR / f"{MUESTRA_REF}_2.
↳fastq.gz"
pares = [("https://" + ftp[0], r1.name), ("https://" + ftp[1], r2.name)]

tiempo_urllib_s = descargar_con_urllib(pares, RAW_DIR)
bytes_urllib = tamano(r1) + tamano(r2)
mbps_urllib = bytes_urllib * 8 / 1e6 / tiempo_urllib_s

print(f"urllib (1 conexion/archivo): {bytes_urllib/1e9:.2f}GB en_
↳{tiempo_urllib_s/60:.1f} min "
      f"-> {mbps_urllib:.2f} Mbps")

```

Rango invalido; reinicio ERR1135179_1.fastq.gz desde cero.

ERR1135179_1.fastq.gz: 100.0% 2.1GB/2.1GB 2.6MB/s ETA 0m00s

Rango invalido; reinicio ERR1135179_2.fastq.gz desde cero.

ERR1135179_2.fastq.gz: 100.0% 2.0GB/2.0GB 2.6MB/s ETA 0m00s

urllib (1 conexion/archivo): 4.40GB en 27.1 min -> 21.63 Mbps

```

[14]: for archivo in (r1, r2):
        archivo.unlink(missing_ok=True)

tiempo_descarga_s, ram_pico_descarga_mb, nucleo_s_descarga, _ =
↳descargar_con_aria2c(
    pares, RAW_DIR, THREADS, bytes_totales_esperados=bytes_totales_esperados)

for archivo, esperado in zip((r1, r2), md5_esperado):
    obtenido = md5sum(archivo)
    if obtenido != esperado:
        raise RuntimeError(f"MD5 no coincide para {archivo.name}: {obtenido} !=_
↳{esperado}")
print(f"Descarga verificada (MD5): {r1.name}, {r2.name}")

bytes_descargados = tamano(r1) + tamano(r2)
velocidad_medida_mbps = bytes_descargados * 8 / 1e6 / tiempo_descarga_s

mediciones.append({
    "paso": "descarga ENA", "tiempo_s": tiempo_descarga_s, "ram_pico_MB":_
↳ram_pico_descarga_mb,

```

```

        "nucleo_segundos": nucleo_s_descarga,
        "disco_bytes": bytes_descargados,
        "nota": (f"velocidad real medida: {velocidad_medida_mbps:.2f} Mbps "
                 f"(aria2c, {min(THREADS, 16)}) conexiones/archivo); comparado con_
↪urllib "
                 f"(1 conexion/archivo): {mbps_urllib:.2f} Mbps"),
    })

print(f"urllib (1 conexion/archivo): {bytes_urllib/1e9:.2f}GB en_
↪{tiempo_urllib_s/60:.1f} min "
      f"-> {mbps_urllib:.2f} Mbps")
print(f"aria2c ({min(THREADS, 16)}) conexiones/archivo): {bytes_descargados/1e9:.
↪2f}GB en "
      f"{tiempo_descarga_s/60:.1f} min -> {velocidad_medida_mbps:.2f} Mbps")

```

```

aria2c: 100.0% 4.1GB/4.1GB 8.8MB/s ETA 0m00s
Descarga verificada (MD5): ERR1135179_1.fastq.gz, ERR1135179_2.fastq.gz
urllib (1 conexion/archivo): 4.40GB en 27.1 min -> 21.63 Mbps
aria2c (12 conexiones/archivo): 4.40GB en 8.0 min -> 73.62 Mbps

```

1.1.2 Control de calidad (fastp) sobre la muestra completa

```

[15]: clean1, clean2 = WORK_DIR / "clean_1.fastq.gz", WORK_DIR / "clean_2.fastq.gz"
cmd = (f"fastp -i {r1} -I {r2} -o {clean1} -O {clean2} -w {THREADS} "
      f"-h {OUT_DIR}/{MUESTRA_REF}_fastp.html -j {OUT_DIR}/{MUESTRA_REF}_fastp.
↪json")
tiempo_s, ram_mb, nucleo_s, _ = sh_medido(cmd)
disco_bytes = tamano(clean1) + tamano(clean2)

fastp_json = json.load(open(f"{OUT_DIR}/{MUESTRA_REF}_fastp.json"))
pares_totales = fastp_json["summary"]["before_filtering"]["total_reads"] // 2
bases_totales = fastp_json["summary"]["before_filtering"]["total_bases"]
pares_post_qc = fastp_json["summary"]["after_filtering"]["total_reads"] // 2

mediciones.append({"paso": "QC (fastp)", "tiempo_s": tiempo_s, "ram_pico_MB":_
↪ram_mb,
                  "nucleo_segundos": nucleo_s, "disco_bytes": disco_bytes,
                  "nota": f"{pares_totales} pares, {bases_totales/1e9:.
↪2f}Gbp"})
print(f"QC: {tiempo_s/60:.1f} min, RAM pico {ram_mb:.0f}MB, {pares_totales}_
↪pares de lecturas")

```

```

[ 1m17s] Read1 before filtering:
[ 1m17s] total reads: 28036112
[ 1m17s] total bases: 2672362430
[ 1m17s] Q20 bases: 2642875468(98.8966%)
[ 1m17s] Q30 bases: 2556945901(95.6811%)
[ 1m17s] Q40 bases: 1041188468(38.9613%)

```

```

[ 1m17s]
[ 1m17s] Read2 before filtering:
[ 1m17s] total reads: 28036112
[ 1m17s] total bases: 2567204363
[ 1m17s] Q20 bases: 2534990644(98.7452%)
[ 1m17s] Q30 bases: 2432771385(94.7634%)
[ 1m17s] Q40 bases: 1010589567(39.3654%)
[ 1m17s]
[ 1m17s] Read1 after filtering:
[ 1m17s] total reads: 28035634
[ 1m17s] total bases: 2671903529
[ 1m17s] Q20 bases: 2642424854(98.8967%)
[ 1m17s] Q30 bases: 2556526052(95.6818%)
[ 1m17s] Q40 bases: 1041087655(38.9643%)
[ 1m17s]
[ 1m17s] Read2 after filtering:
[ 1m17s] total reads: 28035634
[ 1m17s] total bases: 2566811954
[ 1m17s] Q20 bases: 2534620458(98.7459%)
[ 1m17s] Q30 bases: 2432443520(94.7652%)
[ 1m17s] Q40 bases: 1010518223(39.3686%)
[ 1m17s]
[ 1m17s] Filtering result:
[ 1m17s] reads passed filter: 56071268
[ 1m17s] reads failed due to low quality: 956
[ 1m17s] reads failed due to too many N: 0
[ 1m17s] reads failed due to too short: 0
[ 1m17s] reads failed due to adapter dimer: 0
[ 1m17s] reads with adapter trimmed: 26664
[ 1m17s] bases trimmed due to adapters: 784017
[ 1m17s]
[ 1m17s] Duplication rate: 0.448058%
[ 1m17s]
[ 1m17s] Insert size peak (evaluated by paired-end reads): 96
[ 1m17s]
[ 1m17s] JSON report: results/ERR1135179/ERR1135179_fastp.json
[ 1m17s] HTML report: results/ERR1135179/ERR1135179_fastp.html
[ 1m17s]
[ 1m17s] fastp -i raw/ERR1135179/ERR1135179_1.fastq.gz -I
raw/ERR1135179/ERR1135179_2.fastq.gz -o work/ERR1135179/clean_1.fastq.gz -O
work/ERR1135179/clean_2.fastq.gz -w 12 -h
results/ERR1135179/ERR1135179_fastp.html -j
results/ERR1135179/ERR1135179_fastp.json
[ 1m17s] fastp v1.3.6, time used: 77 seconds
[ 1m17s] Command being timed: "fastp -i
raw/ERR1135179/ERR1135179_1.fastq.gz -I raw/ERR1135179/ERR1135179_2.fastq.gz -o
work/ERR1135179/clean_1.fastq.gz -O work/ERR1135179/clean_2.fastq.gz -w 12 -h
results/ERR1135179/ERR1135179_fastp.html -j

```

```

results/ERR1135179/ERR1135179_fastp.json"
[ 1m17s]      User time (seconds): 730.49
[ 1m17s]      System time (seconds): 31.01
[ 1m17s]      Percent of CPU this job got: 979%
[ 1m17s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 1:17.72
[ 1m17s]      Average shared text size (kbytes): 0
[ 1m17s]      Average unshared data size (kbytes): 0
[ 1m17s]      Average stack size (kbytes): 0
[ 1m17s]      Average total size (kbytes): 0
[ 1m17s]      Maximum resident set size (kbytes): 1269480
[ 1m17s]      Average resident set size (kbytes): 0
[ 1m17s]      Major (requiring I/O) page faults: 15
[ 1m17s]      Minor (reclaiming a frame) page faults: 320221
[ 1m17s]      Voluntary context switches: 1151585
[ 1m17s]      Involuntary context switches: 102108
[ 1m17s]      Swaps: 0
[ 1m17s]      File system inputs: 5825336
[ 1m17s]      File system outputs: 8866224
[ 1m17s]      Socket messages sent: 0
[ 1m17s]      Socket messages received: 0
[ 1m17s]      Signals delivered: 0
[ 1m17s]      Page size (bytes): 4096
[ 1m17s]      Exit status: 0
Completado en 1m17s
QC: 1.3 min, RAM pico 1240MB, 28036112 pares de lecturas

```

1.1.3 Submuestreo aleatorio (seqtk)

`seqtk sample` puede leer el `.gz` directo o desde `stdin` ya descomprimido. Antes de quedarnos con un pool para el resto de esta corrida, se compara empíricamente, sobre el mismo `frac_total` y semilla, cómo alimentarlo:

- `zcat + stdin`: descompresión de un solo hilo, vía pipe.
- `pigz + stdin`: descompresión multi-hilo (`-p{THREADS}`), vía pipe.
- `.gz` directo (método usado en el resto del pipeline): `seqtk` descomprime internamente con `zlib` de un solo hilo.

Cada método se corre, se cronometra y se borra su salida antes de correr el siguiente. La salida del último método es la que se conserva y se parte en las `N_SECCIONES` secciones que usa el resto del pipeline. Las tres deben producir exactamente el mismo contenido. Se hizo el cálculo final utilizando la estrategia de mejor rendimiento en este experimento.

```

[20]: LINEAS_POR_LECTURA = 4
frac_total = N_SECCIONES * FRACCION_SECCION
pool1, pool2 = WORK_DIR / "pool_1.fastq", WORK_DIR / "pool_2.fastq"

t_zcat = time.time()
subprocess.run(f"bash -c 'zcat {clean1} | seqtk sample -s{SEED} - {frac_total} \
↪ {pool1}'", shell=True, check=True)

```

```

subprocess.run(f"bash -c 'zcat {clean2} | seqtk sample -s{SEED} - {frac_total} < {pool2}'", shell=True, check=True)
t_zcat = time.time() - t_zcat

hash_zcat = (md5sum(pool1), md5sum(pool2))

pool1.unlink()
pool2.unlink()

print(f"zcat + stdin: {t_zcat:.1f}s")

```

zcat + stdin: 148.9s

```

[21]: t_pigz = time.time()
subprocess.run(f"bash -c 'pigz -dc -p{THREADS} {clean1} | seqtk sample -s{SEED} < {frac_total} > {pool1}'",
               shell=True, check=True)
subprocess.run(f"bash -c 'pigz -dc -p{THREADS} {clean2} | seqtk sample -s{SEED} < {frac_total} > {pool2}'",
               shell=True, check=True)
t_pigz = time.time() - t_pigz

hash_pigz = (md5sum(pool1), md5sum(pool2))
assert hash_pigz == hash_zcat, "pigz y zcat NO produjeron el mismo contenido"

pool1.unlink()
pool2.unlink()

print(f"pigz -p{THREADS} + stdin: {t_pigz:.1f}s")

```

pigz -p12 + stdin: 119.8s

```

[22]: t1_s, ram1_mb, nucleo1_s, _ = sh_medido(f"seqtk sample -s{SEED} {clean1} < {frac_total} > {pool1}")
t2_s, ram2_mb, nucleo2_s, _ = sh_medido(f"seqtk sample -s{SEED} {clean2} < {frac_total} > {pool2}")

hash_directo = (md5sum(pool1), md5sum(pool2))
assert hash_directo == hash_zcat, "seqtk sobre .gz directo NO produjo el mismo contenido que zcat/pigz"
print("Verificado: las 3 formas de alimentar seqtk producen contenido identico (MD5).")

pool_lineas = int(subprocess.run(f"wc -l < {pool1}", shell=True, text=True, capture_output=True).stdout)
pool_pares = pool_lineas // LINEAS_POR_LECTURA

```

```

tiempo_s = t1_s + t2_s
ram_mb = max(ram1_mb, ram2_mb)
nucleo_s = nucleo1_s + nucleo2_s
disco_bytes = tamano(pool1) + tamano(pool2)
mediciones.append({"paso": "submuestreo (seqtk, aleatorio)", "tiempo_s":
    ↪ tiempo_s, "ram_pico_MB": ram_mb,
    "nucleo_segundos": nucleo_s, "disco_bytes": disco_bytes,
    "nota": f"pool aleatorio de {pool_pares:,} pares
    ↪ ({pool_pares/pares_post_qc:.1%} de "
    f"{pares_post_qc:,}, semilla {SEED})"})

print(f"zcat + stdin: {t_zcat:.1f}s")
print(f"pigz -p{THREADS} + stdin: {t_pigz:.1f}s")
print(f".gz directo (produccion): {tiempo_s:.1f}s, RAM pico {ram_mb:.0f}MB,
    ↪ {pool_pares:,} pares reales en el pool")

n_seccion = pool_pares // N_SECCIONES
sub_pares = {}
for i in range(1, N_SECCIONES + 1):
    ini, fin = (i - 1) * n_seccion, i * n_seccion
    ini_linea, n_lineas = ini * LINEAS_POR_LECTURA, n_seccion *
    ↪ LINEAS_POR_LECTURA

    sub1, sub2 = WORK_DIR / f"sub_1_{i}.fastq", WORK_DIR / f"sub_2_{i}.fastq"
    subprocess.run(f"tail -n +{ini_linea + 1} {pool1} | head -n {n_lineas} >
    ↪ {sub1}", shell=True, check=True)
    subprocess.run(f"tail -n +{ini_linea + 1} {pool2} | head -n {n_lineas} >
    ↪ {sub2}", shell=True, check=True)

    sub_pares[i] = (sub1, sub2)
    print(f"Seccion {i}: {fin-ini:,} pares del pool aleatorio (posiciones [{ini:
    ↪ ,}, {fin:,}] dentro del pool)")

pool1.unlink()
pool2.unlink()

```

```

[ 0m48s]          Command being timed: "seqtk sample -s100
work/ERR1135179/clean_1.fastq.gz 0.2"
[ 0m48s]          User time (seconds): 42.28
[ 0m48s]          System time (seconds): 5.98
[ 0m48s]          Percent of CPU this job got: 99%
[ 0m48s]          Elapsed (wall clock) time (h:mm:ss or m:ss): 0:48.64
[ 0m48s]          Average shared text size (kbytes): 0
[ 0m48s]          Average unshared data size (kbytes): 0
[ 0m48s]          Average stack size (kbytes): 0
[ 0m48s]          Average total size (kbytes): 0
[ 0m48s]          Maximum resident set size (kbytes): 2408

```

```

[ 0m48s]      Average resident set size (kbytes): 0
[ 0m48s]      Major (requiring I/O) page faults: 1
[ 0m48s]      Minor (reclaiming a frame) page faults: 122
[ 0m48s]      Voluntary context switches: 106
[ 0m48s]      Involuntary context switches: 231
[ 0m48s]      Swaps: 0
[ 0m48s]      File system inputs: 1338888
[ 0m48s]      File system outputs: 2817672
[ 0m48s]      Socket messages sent: 0
[ 0m48s]      Socket messages received: 0
[ 0m48s]      Signals delivered: 0
[ 0m48s]      Page size (bytes): 4096
[ 0m48s]      Exit status: 0

```

Completado en 0m48s

```

[ 0m46s]      Command being timed: "seqtk sample -s100
work/ERR1135179/clean_2.fastq.gz 0.2"
[ 0m46s]      User time (seconds): 40.56
[ 0m46s]      System time (seconds): 5.66
[ 0m46s]      Percent of CPU this job got: 100%
[ 0m46s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 0:46.16
[ 0m46s]      Average shared text size (kbytes): 0
[ 0m46s]      Average unshared data size (kbytes): 0
[ 0m46s]      Average stack size (kbytes): 0
[ 0m46s]      Average total size (kbytes): 0
[ 0m46s]      Maximum resident set size (kbytes): 2428
[ 0m46s]      Average resident set size (kbytes): 0
[ 0m46s]      Major (requiring I/O) page faults: 0
[ 0m46s]      Minor (reclaiming a frame) page faults: 117
[ 0m46s]      Voluntary context switches: 16
[ 0m46s]      Involuntary context switches: 145
[ 0m46s]      Swaps: 0
[ 0m46s]      File system inputs: 0
[ 0m46s]      File system outputs: 2735528
[ 0m46s]      Socket messages sent: 0
[ 0m46s]      Socket messages received: 0
[ 0m46s]      Signals delivered: 0
[ 0m46s]      Page size (bytes): 4096
[ 0m46s]      Exit status: 0

```

Completado en 0m46s

Verificado: las 3 formas de alimentar seqtk producen contenido identico (MD5).

zcat + stdin: 148.9s

pigz -p12 + stdin: 119.8s

.gz directo (produccion): 94.8s, RAM pico 2MB, 5,608,125 pares reales en el pool
 Seccion 1: 2,804,062 pares del pool aleatorio (posiciones [0, 2,804,062) dentro del pool)

Seccion 2: 2,804,062 pares del pool aleatorio (posiciones [2,804,062, 5,608,124) dentro del pool)

1.1.4 Ensamblado (MEGAHIT) y predicción de genes (prodigal), por sección

Corre una vez por cada una de las N_SECCIONES submuestras (_1, _2).

```
[23]: contigs = {}
for i, (sub1, sub2) in sub_pares.items():
    megahit_out = WORK_DIR / f"megahit_out_{i}"
    shutil.rmtree(megahit_out, ignore_errors=True)
    tiempo_s, ram_mb, nucleo_s, _ = sh_medido(f"megahit -1 {sub1} -2 {sub2} -t_{\
    ↪{THREADS} -o {megahit_out}")
    contigs[i] = megahit_out / "final.contigs.fa"
    disco_bytes = tamano(megahit_out)

    mediciones.append({"paso": f"ensamblado (MEGAHIT) secc.{i}", "tiempo_s":_\
    ↪tiempo_s, "ram_pico_MB": ram_mb,
                      "nucleo_segundos": nucleo_s, "disco_bytes": disco_bytes,\
    ↪"nota": None})
    print(f"Ensamblado seccion {i}: {tiempo_s/60:.1f} min, RAM pico {ram_mb:.\
    ↪0f}MB")
```

```
[ 0m00s] 2026-09-17 01:06:48 - MEGAHIT v1.2.9
[ 0m00s] 2026-09-17 01:06:48 - Using megahit_core with POPCNT and BMI2 support
[ 0m00s] 2026-09-17 01:06:48 - Convert reads to binary library
[ 0m04s] 2026-09-17 01:06:52 - b'INFO  sequence/io/sequence_lib.cpp : 75 -
Lib 0 (/home/jujgomezru/unal/aprendizajeDeMaquina/proyectoFinalML/work/ERR113517
9/sub_1_1.fastq,/home/jujgomezru/unal/aprendizajeDeMaquina/proyectoFinalML/work/
ERR1135179/sub_2_1.fastq): pe, 5608124 reads, 96 max length'
[ 0m04s] 2026-09-17 01:06:52 - b'INFO  utils/utils.h : 152 -
Real: 4.0699\tuser: 3.4648\tsys: 0.9847\tmaxrss: 209344'
[ 0m04s] 2026-09-17 01:06:52 - k-max reset to: 99
[ 0m04s] 2026-09-17 01:06:52 - Start assembly. Number of CPU threads 12
[ 0m04s] 2026-09-17 01:06:52 - k list: 21,29,39,59,79,99
[ 0m04s] 2026-09-17 01:06:52 - Memory used: 7493717606
[ 0m04s] 2026-09-17 01:06:52 - Extract solid (k+1)-mers for k = 21
[ 0m31s] 2026-09-17 01:07:19 - Build graph for k = 21
[ 0m52s] 2026-09-17 01:07:40 - Assemble contigs from SDBG for k = 21
[ 1m59s] 2026-09-17 01:08:47 - Local assembly for k = 21
[ 2m19s] 2026-09-17 01:09:07 - Extract iterative edges from k = 21 to 29
[ 2m24s] 2026-09-17 01:09:12 - Build graph for k = 29
[ 2m40s] 2026-09-17 01:09:28 - Assemble contigs from SDBG for k = 29
[ 3m30s] 2026-09-17 01:10:18 - Local assembly for k = 29
[ 3m55s] 2026-09-17 01:10:43 - Extract iterative edges from k = 29 to 39
[ 3m59s] 2026-09-17 01:10:47 - Build graph for k = 39
[ 4m12s] 2026-09-17 01:11:01 - Assemble contigs from SDBG for k = 39
[ 4m47s] 2026-09-17 01:11:36 - Local assembly for k = 39
[ 5m17s] 2026-09-17 01:12:05 - Extract iterative edges from k = 39 to 59
[ 5m20s] 2026-09-17 01:12:08 - Build graph for k = 59
[ 5m28s] 2026-09-17 01:12:16 - Assemble contigs from SDBG for k = 59
```

```

[ 5m51s] 2026-09-17 01:12:39 - Local assembly for k = 59
[ 6m23s] 2026-09-17 01:13:11 - Extract iterative edges from k = 59 to 79
[ 6m25s] 2026-09-17 01:13:13 - Build graph for k = 79
[ 6m29s] 2026-09-17 01:13:17 - Assemble contigs from SDBG for k = 79
[ 6m49s] 2026-09-17 01:13:37 - Local assembly for k = 79
[ 7m15s] 2026-09-17 01:14:03 - Extract iterative edges from k = 79 to 99
[ 7m15s] 2026-09-17 01:14:03 - Build graph for k = 99
[ 7m18s] 2026-09-17 01:14:06 - Assemble contigs from SDBG for k = 99
[ 7m31s] 2026-09-17 01:14:19 - Merging to output final contigs
[ 7m31s] 2026-09-17 01:14:19 - 35707 contigs, total 30670505 bp, min 200 bp, max
55629 bp, avg 858 bp, N50 1005 bp
[ 7m31s] 2026-09-17 01:14:20 - ALL DONE. Time elapsed: 451.806471 seconds
[ 7m31s]      Command being timed: "megahit -1 work/ERR1135179/sub_1_1.fastq
-2 work/ERR1135179/sub_2_1.fastq -t 12 -o work/ERR1135179/megahit_out_1"
[ 7m31s]      User time (seconds): 4702.81
[ 7m31s]      System time (seconds): 41.54
[ 7m31s]      Percent of CPU this job got: 1049%
[ 7m31s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 7:31.95
[ 7m31s]      Average shared text size (kbytes): 0
[ 7m31s]      Average unshared data size (kbytes): 0
[ 7m31s]      Average stack size (kbytes): 0
[ 7m31s]      Average total size (kbytes): 0
[ 7m31s]      Maximum resident set size (kbytes): 861720
[ 7m31s]      Average resident set size (kbytes): 0
[ 7m31s]      Major (requiring I/O) page faults: 27
[ 7m31s]      Minor (reclaiming a frame) page faults: 2322931
[ 7m31s]      Voluntary context switches: 368317
[ 7m31s]      Involuntary context switches: 38733
[ 7m31s]      Swaps: 0
[ 7m31s]      File system inputs: 39504
[ 7m31s]      File system outputs: 5093016
[ 7m31s]      Socket messages sent: 0
[ 7m31s]      Socket messages received: 0
[ 7m31s]      Signals delivered: 0
[ 7m31s]      Page size (bytes): 4096
[ 7m31s]      Exit status: 0
Completado en 7m31s
Ensamblado seccion 1: 7.5 min, RAM pico 842MB
[ 0m00s] 2026-09-17 01:14:20 - MEGAHIT v1.2.9
[ 0m00s] 2026-09-17 01:14:20 - Using megahit_core with POPCNT and BMI2 support
[ 0m00s] 2026-09-17 01:14:20 - Convert reads to binary library
[ 0m04s] 2026-09-17 01:14:24 - b'INFO  sequence/io/sequence_lib.cpp : 75 -
Lib 0 (/home/jujgomezru/unal/aprendizajeDeMaquina/proyectoFinalML/work/ERR113517
9/sub_1_2.fastq,/home/jujgomezru/unal/aprendizajeDeMaquina/proyectoFinalML/work/
ERR1135179/sub_2_2.fastq): pe, 5608124 reads, 96 max length'
[ 0m04s] 2026-09-17 01:14:24 - b'INFO  utils/utils.h : 152 -
Real: 4.5495\tuser: 3.3178\tsys: 1.4966\tmaxrss: 201316'
[ 0m04s] 2026-09-17 01:14:24 - k-max reset to: 99

```

```

[ 0m04s] 2026-09-17 01:14:24 - Start assembly. Number of CPU threads 12
[ 0m04s] 2026-09-17 01:14:24 - k list: 21,29,39,59,79,99
[ 0m04s] 2026-09-17 01:14:24 - Memory used: 7493717606
[ 0m04s] 2026-09-17 01:14:24 - Extract solid (k+1)-mers for k = 21
[ 0m29s] 2026-09-17 01:14:49 - Build graph for k = 21
[ 0m50s] 2026-09-17 01:15:10 - Assemble contigs from SDBG for k = 21
[ 1m48s] 2026-09-17 01:16:08 - Local assembly for k = 21
[ 2m07s] 2026-09-17 01:16:27 - Extract iterative edges from k = 21 to 29
[ 2m12s] 2026-09-17 01:16:32 - Build graph for k = 29
[ 2m26s] 2026-09-17 01:16:46 - Assemble contigs from SDBG for k = 29
[ 3m09s] 2026-09-17 01:17:29 - Local assembly for k = 29
[ 3m31s] 2026-09-17 01:17:51 - Extract iterative edges from k = 29 to 39
[ 3m34s] 2026-09-17 01:17:54 - Build graph for k = 39
[ 3m45s] 2026-09-17 01:18:05 - Assemble contigs from SDBG for k = 39
[ 4m19s] 2026-09-17 01:18:39 - Local assembly for k = 39
[ 4m49s] 2026-09-17 01:19:09 - Extract iterative edges from k = 39 to 59
[ 4m52s] 2026-09-17 01:19:12 - Build graph for k = 59
[ 4m59s] 2026-09-17 01:19:19 - Assemble contigs from SDBG for k = 59
[ 5m20s] 2026-09-17 01:19:40 - Local assembly for k = 59
[ 5m49s] 2026-09-17 01:20:09 - Extract iterative edges from k = 59 to 79
[ 5m51s] 2026-09-17 01:20:11 - Build graph for k = 79
[ 5m55s] 2026-09-17 01:20:15 - Assemble contigs from SDBG for k = 79
[ 6m10s] 2026-09-17 01:20:30 - Local assembly for k = 79
[ 6m32s] 2026-09-17 01:20:52 - Extract iterative edges from k = 79 to 99
[ 6m33s] 2026-09-17 01:20:53 - Build graph for k = 99
[ 6m35s] 2026-09-17 01:20:55 - Assemble contigs from SDBG for k = 99
[ 6m46s] 2026-09-17 01:21:06 - Merging to output final contigs
[ 6m46s] 2026-09-17 01:21:06 - 32599 contigs, total 26443264 bp, min 200 bp, max
47003 bp, avg 811 bp, N50 922 bp
[ 6m46s] 2026-09-17 01:21:06 - ALL DONE. Time elapsed: 406.681528 seconds
[ 6m46s] Command being timed: "megahit -1 work/ERR1135179/sub_1_2.fastq
-2 work/ERR1135179/sub_2_2.fastq -t 12 -o work/ERR1135179/megahit_out_2"
[ 6m46s] User time (seconds): 4205.87
[ 6m46s] System time (seconds): 36.51
[ 6m46s] Percent of CPU this job got: 1042%
[ 6m46s] Elapsed (wall clock) time (h:mm:ss or m:ss): 6:46.77
[ 6m46s] Average shared text size (kbytes): 0
[ 6m46s] Average unshared data size (kbytes): 0
[ 6m46s] Average stack size (kbytes): 0
[ 6m46s] Average total size (kbytes): 0
[ 6m46s] Maximum resident set size (kbytes): 814872
[ 6m46s] Average resident set size (kbytes): 0
[ 6m46s] Major (requiring I/O) page faults: 0
[ 6m46s] Minor (reclaiming a frame) page faults: 2156170
[ 6m46s] Voluntary context switches: 345008
[ 6m46s] Involuntary context switches: 36689
[ 6m46s] Swaps: 0
[ 6m46s] File system inputs: 0

```

```
[ 6m46s]      File system outputs: 4586824
[ 6m46s]      Socket messages sent: 0
[ 6m46s]      Socket messages received: 0
[ 6m46s]      Signals delivered: 0
[ 6m46s]      Page size (bytes): 4096
[ 6m46s]      Exit status: 0
```

Completado en 6m46s

Ensamblado seccion 2: 6.8 min, RAM pico 796MB

```
[24]: genes_faa, genes_fna, genes_gff = {}, {}, {}
for i in contigs:
    faa, fna = WORK_DIR / f"genes_{i}.faa", WORK_DIR / f"genes_{i}.fna"
    gff = WORK_DIR / f"genes_{i}.gff"
    tiempo_s, ram_mb, nucleo_s, _ = sh_medido(f"prodigal -i {contigs[i]} -a_
↪{faa} -d {fna} -p meta -q -o {gff} -f gff")
    disco_bytes = tamano(faa) + tamano(fna) + tamano(gff)
    n_genes = int(subprocess.run(f"grep -c '>' {faa}", shell=True, text=True,
↪capture_output=True).stdout or 0)

    genes_faa[i], genes_fna[i], genes_gff[i] = faa, fna, gff
    mediciones.append({"paso": f"prediccion de genes (prodigal) secc.{i}",
↪"tiempo_s": tiempo_s, "ram_pico_MB": ram_mb,
                        "nucleo_segundos": nucleo_s, "disco_bytes": disco_bytes,
↪"nota": f"{n_genes} genes candidatos"})
    print(f"Prodigal seccion {i}: {tiempo_s/60:.1f} min, RAM pico {ram_mb:.
↪0f}MB, {n_genes} genes candidatos")
```

```
[ 1m30s]      Command being timed: "prodigal -i
work/ERR1135179/megahit_out_1/final.contigs.fa -a work/ERR1135179/genes_1.faa -d
work/ERR1135179/genes_1.fna -p meta -q -o work/ERR1135179/genes_1.gff -f gff"
[ 1m30s]      User time (seconds): 90.18
[ 1m30s]      System time (seconds): 0.51
[ 1m30s]      Percent of CPU this job got: 99%
[ 1m30s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 1:30.71
[ 1m30s]      Average shared text size (kbytes): 0
[ 1m30s]      Average unshared data size (kbytes): 0
[ 1m30s]      Average stack size (kbytes): 0
[ 1m30s]      Average total size (kbytes): 0
[ 1m30s]      Maximum resident set size (kbytes): 42960
[ 1m30s]      Average resident set size (kbytes): 0
[ 1m30s]      Major (requiring I/O) page faults: 1
[ 1m30s]      Minor (reclaiming a frame) page faults: 7531
[ 1m30s]      Voluntary context switches: 26
[ 1m30s]      Involuntary context switches: 294
[ 1m30s]      Swaps: 0
[ 1m30s]      File system inputs: 24160
[ 1m30s]      File system outputs: 135536
[ 1m30s]      Socket messages sent: 0
```

```

[ 1m30s]      Socket messages received: 0
[ 1m30s]      Signals delivered: 0
[ 1m30s]      Page size (bytes): 4096
[ 1m30s]      Exit status: 0
Completado en 1m30s
Prodigal seccion 1: 1.5 min, RAM pico 42MB, 55034 genes candidatos
[ 1m26s]      Command being timed: "prodigal -i
work/ERR1135179/megahit_out_2/final.contigs.fa -a work/ERR1135179/genes_2.faa -d
work/ERR1135179/genes_2.fna -p meta -q -o work/ERR1135179/genes_2.gff -f gff"
[ 1m26s]      User time (seconds): 85.30
[ 1m26s]      System time (seconds): 0.57
[ 1m26s]      Percent of CPU this job got: 99%
[ 1m26s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 1:26.14
[ 1m26s]      Average shared text size (kbytes): 0
[ 1m26s]      Average unshared data size (kbytes): 0
[ 1m26s]      Average stack size (kbytes): 0
[ 1m26s]      Average total size (kbytes): 0
[ 1m26s]      Maximum resident set size (kbytes): 43096
[ 1m26s]      Average resident set size (kbytes): 0
[ 1m26s]      Major (requiring I/O) page faults: 0
[ 1m26s]      Minor (reclaiming a frame) page faults: 7517
[ 1m26s]      Voluntary context switches: 14
[ 1m26s]      Involuntary context switches: 285
[ 1m26s]      Swaps: 0
[ 1m26s]      File system inputs: 0
[ 1m26s]      File system outputs: 118640
[ 1m26s]      Socket messages sent: 0
[ 1m26s]      Socket messages received: 0
[ 1m26s]      Signals delivered: 0
[ 1m26s]      Page size (bytes): 4096
[ 1m26s]      Exit status: 0
Completado en 1m26s
Prodigal seccion 2: 1.4 min, RAM pico 42MB, 48738 genes candidatos

```

1.1.5 Preparación de bases de datos compartidas (CARD, AMRFinderPlus)

La detección de ARG depende de dos bases de datos compartidas. Las bases de datos se descargaron desde cero. Pero, este paso sólo se realiza una vez al inicio del pipeline, por lo que lo calculamos como una acción individual, y no como un evento cíclico.

```

[25]: CARD_URL = "https://card.mcmaster.ca/latest/data"
LOCALDB_DIR = Path("localDB")
LOCALDB_DIR.mkdir(exist_ok=True)

assert not any(LOCALDB_DIR.iterdir()), (
    "localDB/ no esta vacia -- borrala a mano antes de correr esta seccion,
    para que la descarga y "

```

```

    "carga que se miden abajo sean reales, no una base ya cargada de una
    ↪corrida anterior."
)

t_wget, ram_wget, nucleo_wget, _ = sh_medido(f"wget -q {CARD_URL} -O card_data.
    ↪tar.bz2")
disco_wget = tamano("card_data.tar.bz2")
velocidad_card_mbps = disco_wget * 8 / 1e6 / t_wget
mediciones.append({"paso": "descarga CARD DB", "tiempo_s": t_wget,
    ↪"ram_pico_MB": ram_wget,
    ↪"nucleo_segundos": nucleo_wget, "disco_bytes": disco_wget,
    ↪"nota": f"velocidad real medida: {velocidad_card_mbps:.2f}
    ↪Mbps (wget, 1 conexion)"}))
print(f"Descarga CARD DB: {disco_wget/1e6:.1f}MB en {t_wget:.1f}s ->
    ↪{velocidad_card_mbps:.2f} Mbps")

```

```

[ 0m01s]      Command being timed: "wget -q
https://card.mcmaster.ca/latest/data -O card_data.tar.bz2"
[ 0m01s]      User time (seconds): 0.02
[ 0m01s]      System time (seconds): 0.04
[ 0m01s]      Percent of CPU this job got: 3%
[ 0m01s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 0:01.92
[ 0m01s]      Average shared text size (kbytes): 0
[ 0m01s]      Average unshared data size (kbytes): 0
[ 0m01s]      Average stack size (kbytes): 0
[ 0m01s]      Average total size (kbytes): 0
[ 0m01s]      Maximum resident set size (kbytes): 10848
[ 0m01s]      Average resident set size (kbytes): 0
[ 0m01s]      Major (requiring I/O) page faults: 7
[ 0m01s]      Minor (reclaiming a frame) page faults: 782
[ 0m01s]      Voluntary context switches: 223
[ 0m01s]      Involuntary context switches: 0
[ 0m01s]      Swaps: 0
[ 0m01s]      File system inputs: 25552
[ 0m01s]      File system outputs: 8536
[ 0m01s]      Socket messages sent: 0
[ 0m01s]      Socket messages received: 0
[ 0m01s]      Signals delivered: 0
[ 0m01s]      Page size (bytes): 4096
[ 0m01s]      Exit status: 0
Completado en 0m01s
Descarga CARD DB: 4.4MB en 1.9s -> 18.19 Mbps

```

```

[26]: t_tar, ram_tar, nucleo_tar, _ = sh_medido("tar -xjf card_data.tar.bz2 ./card.
    ↪json")
t_rgoload, ram_rgoload, nucleo_rgoload, _ = sh_medido("rgi load --card_json
    ↪card.json --local")

```

```

assert Path("localDB/card.json").exists(), "CARD no quedo cargada; revisa los
↳pasos de arriba."

tiempo_carga_card_s = t_tar + t_rgiload
ram_carga_card_mb = max(ram_tar, ram_rgiload)
nucleo_carga_card_s = nucleo_tar + nucleo_rgiload
disco_carga_card = tamano("localDB/card.json")
mediciones.append({"paso": "carga/indexado CARD DB (rgi load)", "tiempo_s":
↳tiempo_carga_card_s,
                    "ram_pico_MB": ram_carga_card_mb, "nucleo_segundos":
↳nucleo_carga_card_s,
                    "disco_bytes": disco_carga_card,
                    "nota": "extraccion (tar) + indexado local (rgi load
↳--local) -- CPU, no red"})
print(f"Carga/indexado CARD DB: {tiempo_carga_card_s:.1f}s, RAM pico
↳{ram_carga_card_mb:.0f}MB, "
      f"{disco_carga_card/1e6:.1f}MB en localDB/card.json")

```

```

[ 0m02s]      Command being timed: "tar -xjf card_data.tar.bz2 ./card.json"
[ 0m02s]      User time (seconds): 1.84
[ 0m02s]      System time (seconds): 0.51
[ 0m02s]      Percent of CPU this job got: 108%
[ 0m02s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 0:02.17
[ 0m02s]      Average shared text size (kbytes): 0
[ 0m02s]      Average unshared data size (kbytes): 0
[ 0m02s]      Average stack size (kbytes): 0
[ 0m02s]      Average total size (kbytes): 0
[ 0m02s]      Maximum resident set size (kbytes): 4924
[ 0m02s]      Average resident set size (kbytes): 0
[ 0m02s]      Major (requiring I/O) page faults: 3
[ 0m02s]      Minor (reclaiming a frame) page faults: 1156
[ 0m02s]      Voluntary context switches: 9871
[ 0m02s]      Involuntary context switches: 8
[ 0m02s]      Swaps: 0
[ 0m02s]      File system inputs: 1808
[ 0m02s]      File system outputs: 74392
[ 0m02s]      Socket messages sent: 0
[ 0m02s]      Socket messages received: 0
[ 0m02s]      Signals delivered: 0
[ 0m02s]      Page size (bytes): 4096
[ 0m02s]      Exit status: 0
Completado en 0m02s
[ 0m06s]      Command being timed: "rgi load --card_json card.json --local"
[ 0m06s]      User time (seconds): 4.40
[ 0m06s]      System time (seconds): 2.02
[ 0m06s]      Percent of CPU this job got: 98%
[ 0m06s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 0:06.53

```

```

[ 0m06s]      Average shared text size (kbytes): 0
[ 0m06s]      Average unshared data size (kbytes): 0
[ 0m06s]      Average stack size (kbytes): 0
[ 0m06s]      Average total size (kbytes): 0
[ 0m06s]      Maximum resident set size (kbytes): 393516
[ 0m06s]      Average resident set size (kbytes): 0
[ 0m06s]      Major (requiring I/O) page faults: 367
[ 0m06s]      Minor (reclaiming a frame) page faults: 125652
[ 0m06s]      Voluntary context switches: 3602
[ 0m06s]      Involuntary context switches: 28
[ 0m06s]      Swaps: 0
[ 0m06s]      File system inputs: 651136
[ 0m06s]      File system outputs: 74400
[ 0m06s]      Socket messages sent: 0
[ 0m06s]      Socket messages received: 0
[ 0m06s]      Signals delivered: 0
[ 0m06s]      Page size (bytes): 4096
[ 0m06s]      Exit status: 0

```

Completado en 0m06s

Carga/indexado CARD DB: 8.7s, RAM pico 384MB, 38.1MB en localDB/card.json

```

[27]: AMRFINDERPLUS_DB_DIR = Path("localDB/amrfinderplus")

t_amr, ram_amr, nucleo_amr, _ = sh_medido(f"amrfinder_update -d_
↳{AMRFINDERPLUS_DB_DIR}")
assert (AMRFINDERPLUS_DB_DIR / "latest").exists(), "AMRFinderPlus DB no quedo_
↳lista; revisa el paso de arriba."
disco_amr = tamaño(AMRFINDERPLUS_DB_DIR)
velocidad_amr_mbps = disco_amr * 8 / 1e6 / t_amr
mediciones.append({"paso": "descarga AMRFinderPlus DB", "tiempo_s": t_amr,
↳"ram_pico_MB": ram_amr,
↳"nucleo_segundos": nucleo_amr, "disco_bytes": disco_amr,
↳"nota": f"velocidad real medida: {velocidad_amr_mbps:.2f}_
↳Mbps (amrfinder_update, NCBI FTP)"})
print(f"Descarga AMRFinderPlus DB: {disco_amr/1e6:.1f}MB en {t_amr:.1f}s ->_
↳{velocidad_amr_mbps:.2f} Mbps")

```

```

[ 0m00s] Running: amrfinder_update -d localDB/amrfinderplus
[ 0m00s] Looking up the published databases at https://ftp.ncbi.nlm.nih.gov/path
ogen/Antimicrobial_resistance/AMRFinderPlus/database/
[ 0m01s] Looking for the target directory: localDB/amrfinderplus/2026-08-07.1/
[ 0m01s] Downloading AMRFinder database version 2026-08-07.1 into:
localDB/amrfinderplus/2026-08-07.1/
[ 0m32s] Running: /home/jujgomezru/miniconda3/envs/amr-ml/bin/amrfinder_index
localDB/amrfinderplus/2026-08-07.1/
[ 0m32s] Indexing
[ 0m37s] amrfinder_index took 6 seconds to complete
[ 0m37s] amrfinder_update took 38 seconds to complete

```

```

[ 0m37s]      Command being timed: "amrfinder_update -d localDB/amrfinderplus"
[ 0m37s]      User time (seconds): 5.35
[ 0m37s]      System time (seconds): 1.97
[ 0m37s]      Percent of CPU this job got: 19%
[ 0m37s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 0:37.74
[ 0m37s]      Average shared text size (kbytes): 0
[ 0m37s]      Average unshared data size (kbytes): 0
[ 0m37s]      Average stack size (kbytes): 0
[ 0m37s]      Average total size (kbytes): 0
[ 0m37s]      Maximum resident set size (kbytes): 65016
[ 0m37s]      Average resident set size (kbytes): 0
[ 0m37s]      Major (requiring I/O) page faults: 112
[ 0m37s]      Minor (reclaiming a frame) page faults: 41796
[ 0m37s]      Voluntary context switches: 3613
[ 0m37s]      Involuntary context switches: 55
[ 0m37s]      Swaps: 0
[ 0m37s]      File system inputs: 645208
[ 0m37s]      File system outputs: 494224
[ 0m37s]      Socket messages sent: 0
[ 0m37s]      Socket messages received: 0
[ 0m37s]      Signals delivered: 0
[ 0m37s]      Page size (bytes): 4096
[ 0m37s]      Exit status: 0

```

Completado en 0m37s

Descarga AMRFinderPlus DB: 252.2MB en 37.7s -> 53.46 Mbps

1.1.6 Detección de ARG (rgi contra CARD local), por sección

```

[33]: rgi_df = {}
for i in genes_faa:
    rgi_out = OUT_DIR / f"rgi_{MUESTRA_REF}_{i}"
    tiempo_s, ram_mb, nucleo_s, _ = sh_medido(f"rgi main -i {genes_faa[i]} -o_{rgi_out} -t protein -a DIAMOND --local --clean")
    disco_bytes = tamano(f"{rgi_out}.txt") + tamano(f"{rgi_out}.json")

    rgi_df[i] = pd.read_csv(f"{rgi_out}.txt", sep="\t")
    mediciones.append({"paso": f"deteccion de ARG (rgi) secc.{i}", "tiempo_s": tiempo_s, "ram_pico_MB": ram_mb,
                      "nucleo_segundos": nucleo_s, "disco_bytes": disco_bytes,
                      "nota": f"{len(rgi_df[i])} genes de resistencia"})
    print(f"RGI seccion {i}: {tiempo_s:.1f}s, RAM pico {ram_mb:.0f}MB, {len(rgi_df[i])} genes de resistencia detectados")

```

```

[ 0m12s]      Command being timed: "rgi main -i work/ERR1135179/genes_1.faa -o results/ERR1135179/rgi_ERR1135179_1 -t protein -a DIAMOND --local --clean"
[ 0m12s]      User time (seconds): 34.93
[ 0m12s]      System time (seconds): 3.85
[ 0m12s]      Percent of CPU this job got: 307%

```

```

[ 0m12s] Elapsed (wall clock) time (h:mm:ss or m:ss): 0:12.62
[ 0m12s] Average shared text size (kbytes): 0
[ 0m12s] Average unshared data size (kbytes): 0
[ 0m12s] Average stack size (kbytes): 0
[ 0m12s] Average total size (kbytes): 0
[ 0m12s] Maximum resident set size (kbytes): 318364
[ 0m12s] Average resident set size (kbytes): 0
[ 0m12s] Major (requiring I/O) page faults: 398
[ 0m12s] Minor (reclaiming a frame) page faults: 305466
[ 0m12s] Voluntary context switches: 10646
[ 0m12s] Involuntary context switches: 1365
[ 0m12s] Swaps: 0
[ 0m12s] File system inputs: 788664
[ 0m12s] File system outputs: 101608
[ 0m12s] Socket messages sent: 0
[ 0m12s] Socket messages received: 0
[ 0m12s] Signals delivered: 0
[ 0m12s] Page size (bytes): 4096
[ 0m12s] Exit status: 0

```

Completado en 0m12s

RGI seccion 1: 12.6s, RAM pico 311MB, 29 genes de resistencia detectados

```

[ 0m09s] Command being timed: "rgi main -i work/ERR1135179/genes_2.faa -o
results/ERR1135179/rgi_ERR1135179_2 -t protein -a DIAMOND --local --clean"

```

```

[ 0m09s] User time (seconds): 32.85
[ 0m09s] System time (seconds): 2.78
[ 0m09s] Percent of CPU this job got: 391%
[ 0m09s] Elapsed (wall clock) time (h:mm:ss or m:ss): 0:09.10
[ 0m09s] Average shared text size (kbytes): 0
[ 0m09s] Average unshared data size (kbytes): 0
[ 0m09s] Average stack size (kbytes): 0
[ 0m09s] Average total size (kbytes): 0
[ 0m09s] Maximum resident set size (kbytes): 316544
[ 0m09s] Average resident set size (kbytes): 0
[ 0m09s] Major (requiring I/O) page faults: 0
[ 0m09s] Minor (reclaiming a frame) page faults: 301410
[ 0m09s] Voluntary context switches: 7129
[ 0m09s] Involuntary context switches: 1337
[ 0m09s] Swaps: 0
[ 0m09s] File system inputs: 26176
[ 0m09s] File system outputs: 88352
[ 0m09s] Socket messages sent: 0
[ 0m09s] Socket messages received: 0
[ 0m09s] Signals delivered: 0
[ 0m09s] Page size (bytes): 4096
[ 0m09s] Exit status: 0

```

Completado en 0m09s

RGI seccion 2: 9.1s, RAM pico 309MB, 21 genes de resistencia detectados

1.1.7 Detección de ARG (NCBI AMRFinderPlus), por sección

Corre en paralelo a CARD/RGI, sobre las mismas proteínas predichas por prodigal. Por defecto amrfinder reporta solo genes con Scope=core (AMR), igual que CARD/RGI, para mantener el mismo alcance que CARD.

Se le pasan también el FASTA de nucleótidos del contig (-n) y el GFF que dejó prodigal para que amrfinder reporte posición (Contig id/Start/Stop/Strand).

```
[34]: AMRFINDER_DB = AMRFINDERPLUS_DB_DIR / "latest"

amrfinder_df = {}
for i in genes_faa:
    amr_out = OUT_DIR / f"amrfinder_{MUESTRA_REF}_{i}.tsv"
    tiempo_s, ram_mb, nucleo_s, _ = sh_medido(
        f"amrfinder -p {genes_faa[i]} -n {contigs[i]} -g {genes_gff[i]} -a_
        ↪prodigal "
        f"-d {AMRFINDER_DB} -o {amr_out} --threads {THREADS}")
    disco_bytes = tamaño(amr_out)

    amrfinder_df[i] = pd.read_csv(amr_out, sep="\t")
    mediciones.append({"paso": f"deteccion de ARG (AMRFinderPlus) secc.{i}",
        ↪ "tiempo_s": tiempo_s, "ram_pico_MB": ram_mb,
        "nucleo_segundos": nucleo_s, "disco_bytes": disco_bytes,
        "nota": f"{len(amrfinder_df[i])} genes de resistencia"})
    print(f"AMRFinderPlus seccion {i}: {tiempo_s:.1f}s, RAM pico {ram_mb:.
        ↪0f}MB, "
        f"{len(amrfinder_df[i])} genes de resistencia detectados")
```

```
[ 0m00s] Running: amrfinder -p work/ERR1135179/genes_1.faa -n
work/ERR1135179/megahit_out_1/final.contigs.fa -g work/ERR1135179/genes_1.gff -a
prodigal -d localDB/amrfinderplus/latest -o
results/ERR1135179/amrfinder_ERR1135179_1.tsv --threads 12
[ 0m00s] Software directory: /home/jujgomezru/miniconda3/envs/amr-ml/bin/
[ 0m00s] Software version: 4.2.7
[ 0m00s] Database directory: /home/jujgomezru/unal/aprendizajeDeMaquina/proyecto
FinalML/localDB/amrfinderplus/2026-08-07.1
[ 0m00s] Database version: 2026-08-07.1
[ 0m00s] AMRFinder combined translated and protein search
[ 0m00s] - include -O ORGANISM, --organism ORGANISM option to add mutation
searches and suppress common proteins
[ 0m03s] Running blastp
[ 0m23s] Running hmmsearch
[ 0m47s] Running blastx
[ 2m17s] Making report
[ 2m19s] amrfinder took 140 seconds to complete
[ 2m19s] Command being timed: "amrfinder -p work/ERR1135179/genes_1.faa
-n work/ERR1135179/megahit_out_1/final.contigs.fa -g work/ERR1135179/genes_1.gff
-a prodigal -d localDB/amrfinderplus/latest -o
```

```

results/ERR1135179/amrfinder_ERR1135179_1.tsv --threads 12"
[ 2m19s]      User time (seconds): 1438.31
[ 2m19s]      System time (seconds): 56.54
[ 2m19s]      Percent of CPU this job got: 1074%
[ 2m19s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 2:19.14
[ 2m19s]      Average shared text size (kbytes): 0
[ 2m19s]      Average unshared data size (kbytes): 0
[ 2m19s]      Average stack size (kbytes): 0
[ 2m19s]      Average total size (kbytes): 0
[ 2m19s]      Maximum resident set size (kbytes): 487488
[ 2m19s]      Average resident set size (kbytes): 0
[ 2m19s]      Major (requiring I/O) page faults: 113
[ 2m19s]      Minor (reclaiming a frame) page faults: 8992275
[ 2m19s]      Voluntary context switches: 16177
[ 2m19s]      Involuntary context switches: 41318
[ 2m19s]      Swaps: 0
[ 2m19s]      File system inputs: 892984
[ 2m19s]      File system outputs: 132744
[ 2m19s]      Socket messages sent: 0
[ 2m19s]      Socket messages received: 0
[ 2m19s]      Signals delivered: 0
[ 2m19s]      Page size (bytes): 4096
[ 2m19s]      Exit status: 0
Completado en 2m19s
AMRFinderPlus seccion 1: 139.1s, RAM pico 476MB, 23 genes de resistencia
detectados
[ 0m00s] Running: amrfinder -p work/ERR1135179/genes_2.faa -n
work/ERR1135179/megahit_out_2/final.contigs.fa -g work/ERR1135179/genes_2.gff -a
prodigal -d localDB/amrfinderplus/latest -o
results/ERR1135179/amrfinder_ERR1135179_2.tsv --threads 12
[ 0m00s] Software directory: /home/jujgomezru/miniconda3/envs/amr-ml/bin/
[ 0m00s] Software version: 4.2.7
[ 0m00s] Database directory: /home/jujgomezru/unal/aprendizajeDeMaquina/proyecto
FinalML/localDB/amrfinderplus/2026-08-07.1
[ 0m00s] Database version: 2026-08-07.1
[ 0m00s] AMRFinder combined translated and protein search
[ 0m00s]   - include -O ORGANISM, --organism ORGANISM option to add mutation
searches and suppress common proteins
[ 0m02s] Running blastp
[ 0m19s] Running hmmsearch
[ 0m40s] Running blastx
[ 1m57s] Making report
[ 1m59s] amrfinder took 119 seconds to complete
[ 1m59s]      Command being timed: "amrfinder -p work/ERR1135179/genes_2.faa
-n work/ERR1135179/megahit_out_2/final.contigs.fa -g work/ERR1135179/genes_2.gff
-a prodigal -d localDB/amrfinderplus/latest -o
results/ERR1135179/amrfinder_ERR1135179_2.tsv --threads 12"
[ 1m59s]      User time (seconds): 1244.25

```

```

[ 1m59s]      System time (seconds): 40.41
[ 1m59s]      Percent of CPU this job got: 1077%
[ 1m59s]      Elapsed (wall clock) time (h:mm:ss or m:ss): 1:59.18
[ 1m59s]      Average shared text size (kbytes): 0
[ 1m59s]      Average unshared data size (kbytes): 0
[ 1m59s]      Average stack size (kbytes): 0
[ 1m59s]      Average total size (kbytes): 0
[ 1m59s]      Maximum resident set size (kbytes): 471448
[ 1m59s]      Average resident set size (kbytes): 0
[ 1m59s]      Major (requiring I/O) page faults: 1
[ 1m59s]      Minor (reclaiming a frame) page faults: 6074482
[ 1m59s]      Voluntary context switches: 14298
[ 1m59s]      Involuntary context switches: 36213
[ 1m59s]      Swaps: 0
[ 1m59s]      File system inputs: 89968
[ 1m59s]      File system outputs: 113024
[ 1m59s]      Socket messages sent: 0
[ 1m59s]      Socket messages received: 0
[ 1m59s]      Signals delivered: 0
[ 1m59s]      Page size (bytes): 4096
[ 1m59s]      Exit status: 0

```

Completado en 1m59s

AMRFinderPlus seccion 2: 119.2s, RAM pico 460MB, 22 genes de resistencia detectados

1.1.8 Cruce CARD vs. AMRFinderPlus, por gen y por familia/clase

CARD (Best_Hit_ARO, p. ej. TEM-181) y AMRFinderPlus (Element symbol, p. ej. blaTEM-181) no usan la misma nomenclatura. Este cruce es heurístico y sujeto a revisión más detallada.

- Por gen: normaliza ambos símbolos y los considera el mismo gen si uno contiene al otro, así TEM181 (CARD) y BLATEM181 (AMRFinderPlus) cruzan pese al prefijo de familia (bla) que CARD no siempre incluye.
- Por familia/clase: mismo criterio de contención, pero entre Drug Class de CARD (texto libre, p. ej. cephalosporin;penicillin beta-lactam) y Class de AMRFinderPlus (categórico, p. ej. BETA-LACTAM), una clasificación más gruesa, útil cuando el cruce por gen específico no encuentra nada pero ambas herramientas sí están mirando la misma familia de resistencia.

```

[35]: def normalizar_gen(nombre):
        return re.sub(r"^[A-Z0-9]", "", str(nombre).upper())

def _partes_gen(nombre):
    n = normalizar_gen(nombre)
    m = re.match(r"^[A-Z]*([0-9].*)?$", n)
    return m.group(1), m.group(2) or ""

```

```

def mismo_gen(a, b, min_len=3):
    pa, na = _partes_gen(a)
    pb, nb = _partes_gen(b)
    if not pa or not pb:
        return False
    if na != nb:
        return False
    if len(pa) < min_len or len(pb) < min_len:
        return pa == pb
    return pa in pb or pb in pa

def cruzar_card_amrfinder(rgi, amrfinder):
    genes_card = sorted(rgi["Best_Hit_ARO"].dropna().unique())
    genes_amr = sorted(amrfinder["Element symbol"].dropna().unique())

    filas, vistos_amr = [], set()
    for g_card in genes_card:
        candidatos = [g for g in genes_amr if mismo_gen(g_card, g)]
        vistos_amr.update(candidatos)
        filas.append({"gen": g_card, "en_CARD": True, "en_AMRFinderPlus":
↪bool(candidatos),
                        "gen_AMRFinderPlus": ", ".join(candidatos) or None})
    for g_amr in genes_amr:
        if g_amr not in vistos_amr:
            filas.append({"gen": g_amr, "en_CARD": False, "en_AMRFinderPlus":
↪True, "gen_AMRFinderPlus": g_amr})
    cruce_gen = pd.DataFrame(filas)

    familias_card = sorted(rgi["Drug Class"].dropna().str.split(";").explode().
↪str.strip().unique())
    clases_amr = sorted(amrfinder["Class"].dropna().unique())

    filas_fam, vistas_amr = [], set()
    for f_card in familias_card:
        candidatas = [c for c in clases_amr if mismo_gen(f_card, c)]
        vistas_amr.update(candidatas)
        filas_fam.append({"familia": f_card, "en_CARD": True,
↪"en_AMRFinderPlus": bool(candidatas)})
    for c_amr in clases_amr:
        if c_amr not in vistas_amr:
            filas_fam.append({"familia": c_amr, "en_CARD": False,
↪"en_AMRFinderPlus": True})
    cruce_familia = pd.DataFrame(filas_fam)

```

```

    return cruce_gen, cruce_familia

cruce_gen, cruce_familia = {}, {}
for i in genes_faa:
    cg, cf = cruzar_card_amrfinder(rgi_df[i], amrfinder_df[i])
    cruce_gen[i], cruce_familia[i] = cg, cf

    ambos = (cg["en_CARD"] & cg["en_AMRFinderPlus"]).sum()
    solo_card = (cg["en_CARD"] & ~cg["en_AMRFinderPlus"]).sum()
    solo_amr = (~cg["en_CARD"] & cg["en_AMRFinderPlus"]).sum()
    print(f"Seccion {i} -- genes: {ambos} en ambas, {solo_card} solo CARD, {solo_amr} solo AMRFinderPlus "
          f"({len(cf)} familias/clases distintas entre las dos)")

cruce_gen[1]

```

Seccion 1 -- genes: 11 en ambas, 12 solo CARD, 12 solo AMRFinderPlus (18 familias/clases distintas entre las dos)

Seccion 2 -- genes: 10 en ambas, 7 solo CARD, 12 solo AMRFinderPlus (14 familias/clases distintas entre las dos)

```

[35]:
0          gen  en_CARD  \
1          ACI-1      True
2    APH(3')-IIIa    True
3          CfxA2      True
4          CfxA6      True
5          ErmB      True
6          ErmF      True
7          ErmG      True
8          Mef(En2)    True
9          MrmA      True
10 Mycobacterium tuberculosis rpoC mutations conf... True
11          SAT-4      True
12          lnuC      True
13          nimJ      True
14          poxtA      True
15          tet(32)    True
16          tet(44)    True
17          tet(0)     True
18          tet(Q)     True
19          tet(W)     True
20          vanT gene in vanG cluster    True
21          vanXY gene in vanG cluster    True
22          vanY gene in vanA cluster     True
23          vanY gene in vanM cluster     True
24          aad9      False

```

24		aadE	False
25		ant(6)	False
26		ant(6)-Ia	False
27		ant(6)-Ib	False
28		blaACI	False
29		cfxA	False
30		estT	False
31		lnu(AN2)	False
32		lsa	False
33		mef(A)	False
34		tet(40)	False

	en_AMRFinderPlus	gen_AMRFinderPlus
0	False	NaN
1	True	aph(3')-IIIa
2	False	NaN
3	True	cfxA6
4	True	erm(B)
5	True	erm(F)
6	True	erm(G)
7	True	mef(En2)
8	False	NaN
9	False	NaN
10	True	sat4
11	True	lnu(C)
12	True	nimJ
13	False	NaN
14	False	NaN
15	True	tet(44)
16	False	NaN
17	True	tet(Q)
18	False	NaN
19	False	NaN
20	False	NaN
21	False	NaN
22	False	NaN
23	True	aad9
24	True	aadE
25	True	ant(6)
26	True	ant(6)-Ia
27	True	ant(6)-Ib
28	True	blaACI
29	True	cfxA
30	True	estT
31	True	lnu(AN2)
32	True	lsa
33	True	mef(A)

1.1.9 Posición de cada ARG dentro de su contig (CARD/RGI y AMRFinderPlus)

Cada ARG ya trae coordenadas de nucleótido dentro de su contig. RGI las toma del encabezado que deja prodigal en genes.faa; AMRFinderPlus las reporta directamente (Contig id/Start/Stop/Strand). Como un metagenoma ensamblado no tiene una única coordenada “de la muestra” (cada contig es independiente), se añade el largo de cada contig y una posición relativa (0 = inicio del contig, 1 = final), por sección y por herramienta.

```
[36]: def contig_de_orf(orf, asm_ids):
    orf = str(orf).split()[0]
    if orf in asm_ids:
        return orf
    partes = orf.split("_")
    for corte in range(len(partes) - 1, 0, -1):
        cand = "_".join(partes[:corte])
        if cand in asm_ids:
            return cand
    return None

def coords_desde_header_prodigal(orf_id):
    m = re.search(r"#s*(\d+)\s*#\s*(\d+)\s*#\s*(-?1)\s*#", str(orf_id))
    if not m:
        return pd.Series([None, None, None])
    inicio, fin, hebra = m.groups()
    return pd.Series([int(inicio), int(fin), int(hebra)])

def agregar_posicion_relativa(df, contigs_path, col_contig, col_start,
    ↪col_stop, resolver_contig=None):
    largos = {r.id: len(r.seq) for r in SeqIO.parse(str(contigs_path), "fasta")}
    df = df.copy()
    if resolver_contig:
        asm_ids = set(largos)
        df["contig_asm"] = df[col_contig].map(lambda x: resolver_contig(x,
    ↪asm_ids))
    else:
        df["contig_asm"] = df[col_contig]
    df["contig_len"] = df["contig_asm"].map(largos)
    df["pos_inicio_rel"] = df[col_start] / df["contig_len"]
    df["pos_fin_rel"] = df[col_stop] / df["contig_len"]
    return df

for i in genes_faa:
```

```

# rgi main -t protein deja Start/Stop/Orientation vacios (no calcula
↳ coordenadas genómicas en ese
# modo) -- se sacan del propio ORF_ID, que trae el header de prodigal
↳ ("contig # inicio # fin # hebra # ...")
rgi_df[i][["Start", "Stop", "Orientation"]] = rgi_df[i]["ORF_ID"].
↳ apply(coords_desde_header_prodigal)

rgi_df[i] = agregar_posicion_relativa(rgi_df[i], contigs[i], "ORF_ID",
↳ "Start", "Stop",
resolver_contig=contig_de_orf)
amrfinder_df[i] = agregar_posicion_relativa(amrfinder_df[i], contigs[i],
↳ "Contig id", "Start", "Stop")

sin_contig_rgi = rgi_df[i]["contig_asm"].isna().sum()
sin_coords_rgi = rgi_df[i]["Start"].isna().sum()
print(f"Seccion {i}: posicion añadida -- RGI {len(rgi_df[i])} -
↳ sin_contig_rgi}/{len(rgi_df[i])} "
f"genes con contig identificado ({len(rgi_df[i]) - sin_coords_rgi}/
↳ {len(rgi_df[i])} con "
f"coordenadas parseadas del header), AMRFinderPlus
↳ {len(amrfinder_df[i])}/{len(amrfinder_df[i])}")

rgi_df[1][["ORF_ID", "contig_asm", "Start", "Stop", "Orientation", "contig_len",
"pos_inicio_rel", "pos_fin_rel", "Best_Hit_ARO"]]

```

Seccion 1: posicion añadida -- RGI 29/29 genes con contig identificado (29/29 con coordenadas parseadas del header), AMRFinderPlus 23/23

Seccion 2: posicion añadida -- RGI 21/21 genes con contig identificado (21/21 con coordenadas parseadas del header), AMRFinderPlus 22/22

[36]:

		ORF_ID	contig_asm	Start	Stop	\
0	k99_36_1 # 291 # 1091 # -1 # ID=308_1;partial=...	k99_36	291	1091		
1	k99_9231_3 # 800 # 1642 # 1 # ID=1890_3;partia...	k99_9231	800	1642		
2	k99_226_1 # 867 # 1832 # 1 # ID=2414_1;partial...	k99_226	867	1832		
3	k99_364_2 # 430 # 1167 # 1 # ID=4066_2;partial...	k99_364	430	1167		
4	k99_24593_5 # 4295 # 5257 # -1 # ID=4161_5;par...	k99_24593	4295	5257		
5	k99_15502_2 # 738 # 1721 # -1 # ID=4663_2;part...	k99_15502	738	1721		
6	k99_30804_2 # 243 # 1121 # -1 # ID=6235_2;part...	k99_30804	243	1121		
7	k99_15670_2 # 428 # 2347 # -1 # ID=6646_2;part...	k99_15670	428	2347		
8	k99_18802_3 # 1136 # 3058 # 1 # ID=7574_3;part...	k99_18802	1136	3058		
9	k99_33927_1 # 2 # 2101 # 1 # ID=7604_1;partial...	k99_33927	2	2101		
10	k99_33939_1 # 2 # 613 # -1 # ID=7737_1;partial...	k99_33939	2	613		
11	k99_19169_3 # 741 # 1946 # -1 # ID=12186_3;par...	k99_19169	741	1946		
12	k99_7157_2 # 101 # 1408 # -1 # ID=12851_2;part...	k99_7157	101	1408		
13	k99_10233_1 # 3 # 659 # 1 # ID=12950_1;partial...	k99_10233	3	659		
14	k99_22531_1 # 110 # 760 # -1 # ID=15852_1;part...	k99_22531	110	760		
15	k99_7379_1 # 2 # 613 # -1 # ID=15999_1;partial...	k99_7379	2	613		

16	k99_13684_1 # 209 # 703 # -1 # ID=19024_1;part...	k99_13684	209	703
17	k99_13901_1 # 2 # 532 # -1 # ID=21657_1;partia...	k99_13901	2	532
18	k99_20207_2 # 432 # 929 # -1 # ID=24319_2;part...	k99_20207	432	929
19	k99_20214_2 # 111 # 956 # -1 # ID=24412_2;part...	k99_20214	111	956
20	k99_26556_1 # 40 # 693 # 1 # ID=26346_1;partia...	k99_26556	40	693
21	k99_29534_1 # 3 # 863 # 1 # ID=27353_1;partial...	k99_29534	3	863
22	k99_35535_1 # 3 # 467 # 1 # ID=27544_1;partial...	k99_35535	3	467
23	k99_35535_2 # 560 # 1354 # 1 # ID=27544_2;part...	k99_35535	560	1354
24	k99_11613_2 # 1331 # 3820 # -1 # ID=28979_2;pa...	k99_11613	1331	3820
25	k99_36045_1 # 88 # 921 # -1 # ID=33585_1;parti...	k99_36045	88	921
26	k99_2904_1 # 1 # 579 # 1 # ID=33715_1;partial=...	k99_2904	1	579
27	k99_21085_1 # 33 # 767 # 1 # ID=34926_1;partia...	k99_21085	33	767
28	k99_23118_1 # 2 # 3418 # -1 # ID=23142_1;parti...	k99_23118	2	3418

	Orientation	contig_len	pos_inicio_rel	pos_fin_rel \
0	-1	1229	0.236778	0.887714
1	1	1643	0.486914	0.999391
2	1	5273	0.164423	0.347430
3	1	7674	0.056033	0.152072
4	-1	5872	0.731437	0.895266
5	-1	3356	0.219905	0.512813
6	-1	1123	0.216385	0.998219
7	-1	2347	0.182360	1.000000
8	1	3365	0.337593	0.908767
9	1	5023	0.000398	0.418276
10	-1	615	0.003252	0.996748
11	-1	2298	0.322454	0.846823
12	-1	1448	0.069751	0.972376
13	1	697	0.004304	0.945481
14	-1	761	0.144547	0.998686
15	-1	615	0.003252	0.996748
16	-1	2150	0.097209	0.326977
17	-1	533	0.003752	0.998124
18	-1	1043	0.414190	0.890700
19	-1	1769	0.062747	0.540418
20	1	693	0.057720	1.000000
21	1	865	0.003468	0.997688
22	1	1807	0.001660	0.258439
23	1	1807	0.309906	0.749308
24	-1	8832	0.150702	0.432518
25	-1	923	0.095341	0.997833
26	1	581	0.001721	0.996558
27	1	1437	0.022965	0.533751
28	-1	9192	0.000218	0.371845

	Best_Hit_ARO
0	ErmF

```

1          tet(0)
2          CfxA2
3          ErmB
4          CfxA6
5          MrmA
6          tet(W)
7          tet(Q)
8          tet(44)
9          vanT gene in vanG cluster
10         tet(32)
11         Mef(En2)
12         vanY gene in vanA cluster
13         tet(0)
14         ACI-1
15         tet(0)
16         lnuC
17         vanXY gene in vanG cluster
18         nimJ
19         vanY gene in vanM cluster
20         tet(W)
21         vanT gene in vanG cluster
22         SAT-4
23         APH(3')-IIIa
24         vanT gene in vanG cluster
25         poxtA
26         tet(Q)
27         ErmG
28 Mycobacterium tuberculosis rpoC mutations conf...

```

1.1.10 Secuencia de aminoácidos de cada ARG detectado

La proteína que sostiene cada ARG ya salió de prodigal previamente, así que el ID de proteína que cada herramienta reporta (ORF_ID en CARD/RGI, Protein id en AMRFinderPlus) coincide exactamente con el encabezado de una secuencia de ese archivo.

Se añade como columna nueva (secuencia_aa) a los mismos rgi_df/amrfinder_df, para poder entrenar tanto con la secuencia de ADN del ARG (ya disponible) como con su secuencia de aminoácidos.

```

[39]: def agregar_secuencia_proteina(df, faa_path, col_id):
        proteinas = {r.id: str(r.seq) for r in SeqIO.parse(str(faa_path), "fasta")}
        df = df.copy()
        df["secuencia_aa"] = df[col_id].map(lambda x: proteinas.get(str(x).
↪split()[0]))
        return df

for i in genes_faa:

```

```

rgi_df[i] = agregar_secuencia_proteina(rgi_df[i], genes_faa[i], "ORF_ID")
amrfinder_df[i] = agregar_secuencia_proteina(amrfinder_df[i], genes_faa[i],
↪ "Protein id")

sin_secuencia_rgi = rgi_df[i]["secuencia_aa"].isna().sum()
sin_secuencia_amr = amrfinder_df[i]["secuencia_aa"].isna().sum()
print(f"Seccion {i}: secuencia de aminoacidos añadida -- "
      f"RGI {len(rgi_df[i]) - sin_secuencia_rgi}/{len(rgi_df[i])}, "
      f"AMRFinderPlus {len(amrfinder_df[i]) - sin_secuencia_amr}/
↪ {len(amrfinder_df[i])}")

rgi_df[1][["ORF_ID", "Best_Hit_ARO", "contig_asm", "pos_inicio_rel",
↪ "pos_fin_rel", "secuencia_aa"]]

```

Seccion 1: secuencia de aminoacidos añadida -- RGI 29/29, AMRFinderPlus 23/23

Seccion 2: secuencia de aminoacidos añadida -- RGI 21/21, AMRFinderPlus 21/22

```

[39]:
ORF_ID \
0 k99_36_1 # 291 # 1091 # -1 # ID=308_1;partial=...
1 k99_9231_3 # 800 # 1642 # 1 # ID=1890_3;partia...
2 k99_226_1 # 867 # 1832 # 1 # ID=2414_1;partial...
3 k99_364_2 # 430 # 1167 # 1 # ID=4066_2;partial...
4 k99_24593_5 # 4295 # 5257 # -1 # ID=4161_5;par...
5 k99_15502_2 # 738 # 1721 # -1 # ID=4663_2;part...
6 k99_30804_2 # 243 # 1121 # -1 # ID=6235_2;part...
7 k99_15670_2 # 428 # 2347 # -1 # ID=6646_2;part...
8 k99_18802_3 # 1136 # 3058 # 1 # ID=7574_3;part...
9 k99_33927_1 # 2 # 2101 # 1 # ID=7604_1;partial...
10 k99_33939_1 # 2 # 613 # -1 # ID=7737_1;partial...
11 k99_19169_3 # 741 # 1946 # -1 # ID=12186_3;par...
12 k99_7157_2 # 101 # 1408 # -1 # ID=12851_2;part...
13 k99_10233_1 # 3 # 659 # 1 # ID=12950_1;partial...
14 k99_22531_1 # 110 # 760 # -1 # ID=15852_1;part...
15 k99_7379_1 # 2 # 613 # -1 # ID=15999_1;partial...
16 k99_13684_1 # 209 # 703 # -1 # ID=19024_1;part...
17 k99_13901_1 # 2 # 532 # -1 # ID=21657_1;partia...
18 k99_20207_2 # 432 # 929 # -1 # ID=24319_2;part...
19 k99_20214_2 # 111 # 956 # -1 # ID=24412_2;part...
20 k99_26556_1 # 40 # 693 # 1 # ID=26346_1;partia...
21 k99_29534_1 # 3 # 863 # 1 # ID=27353_1;partial...
22 k99_35535_1 # 3 # 467 # 1 # ID=27544_1;partial...
23 k99_35535_2 # 560 # 1354 # 1 # ID=27544_2;part...
24 k99_11613_2 # 1331 # 3820 # -1 # ID=28979_2;pa...
25 k99_36045_1 # 88 # 921 # -1 # ID=33585_1;parti...
26 k99_2904_1 # 1 # 579 # 1 # ID=33715_1;partial=...
27 k99_21085_1 # 33 # 767 # 1 # ID=34926_1;partia...
28 k99_23118_1 # 2 # 3418 # -1 # ID=23142_1;parti...

```

	Best_Hit_ARO	contig_asm	\
0	ErmF	k99_36	
1	tet(O)	k99_9231	
2	CfxA2	k99_226	
3	ErmB	k99_364	
4	CfxA6	k99_24593	
5	MrmA	k99_15502	
6	tet(W)	k99_30804	
7	tet(Q)	k99_15670	
8	tet(44)	k99_18802	
9	vanT gene in vanG cluster	k99_33927	
10	tet(32)	k99_33939	
11	Mef(En2)	k99_19169	
12	vanY gene in vanA cluster	k99_7157	
13	tet(O)	k99_10233	
14	ACI-1	k99_22531	
15	tet(O)	k99_7379	
16	lnuC	k99_13684	
17	vanXY gene in vanG cluster	k99_13901	
18	nimJ	k99_20207	
19	vanY gene in vanM cluster	k99_20214	
20	tet(W)	k99_26556	
21	vanT gene in vanG cluster	k99_29534	
22	SAT-4	k99_35535	
23	APH(3')-IIIa	k99_35535	
24	vanT gene in vanG cluster	k99_11613	
25	poxA	k99_36045	
26	tet(Q)	k99_2904	
27	ErmG	k99_21085	
28	Mycobacterium tuberculosis rpoC mutations conf...	k99_23118	

	pos_inicio_rel	pos_fin_rel	\
0	0.236778	0.887714	
1	0.486914	0.999391	
2	0.164423	0.347430	
3	0.056033	0.152072	
4	0.731437	0.895266	
5	0.219905	0.512813	
6	0.216385	0.998219	
7	0.182360	1.000000	
8	0.337593	0.908767	
9	0.000398	0.418276	
10	0.003252	0.996748	
11	0.322454	0.846823	
12	0.069751	0.972376	
13	0.004304	0.945481	

14	0.144547	0.998686
15	0.003252	0.996748
16	0.097209	0.326977
17	0.003752	0.998124
18	0.414190	0.890700
19	0.062747	0.540418
20	0.057720	1.000000
21	0.003468	0.997688
22	0.001660	0.258439
23	0.309906	0.749308
24	0.150702	0.432518
25	0.095341	0.997833
26	0.001721	0.996558
27	0.022965	0.533751
28	0.000218	0.371845

secuencia_aa

0	MTKKKL PVRFTGQHFTIDKVLIKDAIRQANISNQDTVLDIGAGKGF...
1	MKIINLGILAHVDAGKTTLTESLLYTSGAIAELGSVDEGTTRTDTM...
2	MGKNRKKQIVVLCIALVCIFILVFSLFHKSATKDSANPPLTNVLT...
3	MNKNIKYSQNFLTSEKVLNQIIKQLNLKETDTVYEIGTGKGHLLTK...
4	MKKNRKKQIVVLCIALVCIFILVFSLSHKSATKGSANPPLTDVLT...
5	MKRLPKYTPAEVRNDPYGFTYKEMSEVIGENEAKALYEELYKQLPR...
6	TTIAPKTAQAQRERLLDALTLQADTDPLLRCEVDSITHEIILSFLGR...
7	IINLGILAHIDAGKTSVTENLLFASGATEKCGRVDNGDTITDSMDI...
8	MKIINIGILAHVDAGKTTLTESLLYTSGAILELGSVDKGTTRTDTM...
9	HQLLSPNMFVTRSPRSYNSQIGVPLSVWLMNEQTEVGVEAGISQP...
10	CGQVFKIEYSEKRRRFVYVRIYSGTLHLRDVIKISEKEKIKITEMC...
11	MNHWKSTLAVIGIGQLISILTSTIVGFSIIFWISNEFKSPTALSLA...
12	MNNLILVNKNRLEKNYIPSGLIEISDFVEGDIVDYKLKINYNAYE...
13	ERPLRKAETYTIHIEVPPNPFWASVGLSIEPLPIGSGVQYESRVSLG...
14	KVFTAAELLRQKNTSDLNEIRKFS AEDILSYAPITKDHVADGMTLA...
15	CGQVFKIEYSEKRRRFVYVRIYSGTLHLRDVIRISEKEKIKITEMC...
16	MVNITDVKQILQFAIDAEIKVFLDGGWGDALLGYQSRANDIDIF...
17	ATHLSQVMEKINGWQQIVPVSGWRSLEQQDIFKQSLKDNGRFTTQ...
18	MSEFREMRKRQQLSEEEESIAILQKATAGTLALLGDNDYPYAVPIS...
19	MHKQIKKSLLLFSVLLIVSFTACTKSDSKSKQKESASAAQGNLQPE...
20	MFLERQRGITIQAAVTSFQWHRCKVNIVDTPGHMDFLAEVYRSLAV...
21	VKIAHKLKDLPYLFGFAVATAEESFELREAGITKPILILGYTFKED...
22	IPRYENENWTFTELLYEAPYLKSYQDEEDEDEEADCLEYIDNTDK...
23	MAKMRIPELKKLIEKYRCVKDTEGMSPAKVYKLVGENENLYLKMT...
24	MIYSIEKITTLIGARRIGNTDAQIGWLLTDSRSLCFPEESLFFALQ...
25	KQEKEIKELQAFIDKAKKASRTNHNKKMGKDREVKLAKKLEET...
26	MFNIGINELLDIAISSFILPPESVSNRLSAYLYKIEHDPKGHKRSFL...
27	MNKVNIKDSQNFITSKYHIEKIMNCISLDEKDNIFEIGAGKGHFTA...
28	MQIGLASPNKIRSWSYGEVKKPETINYRTLKPEKDGLFDERIFGPT...

1.1.11 Comparación directa: solo CARD, solo AMRFinderPlus, método mixto

Esta sección solo reagrupa esas tres piezas en dos comparaciones directas:

- Eficiencia: tiempo, RAM pico y núcleo-segundos de correr solo CARD, solo AMRFinderPlus, y el método mixto (las dos, que es simplemente la suma de tiempo/núcleo-segundos de ambas más el pico de RAM de ambas).
- Riqueza: genes y familias/clases distintas que encuentra cada método.

Se reporta por sección (N_SECCIONES filas por método) en vez de sumar entre secciones, porque un mismo gen detectado en las dos secciones no debería contarse dos veces al comparar riqueza.

```
[40]: filas_efic, filas_riq = [], []
for i in genes_faa:
    m_card = next(m for m in mediciones if m["paso"] == f"deteccion de ARG_
    ↪(rgi) secc.{i}")
    m_amr = next(m for m in mediciones if m["paso"] == f"deteccion de ARG_
    ↪(AMRFinderPlus) secc.{i}")

    filas_efic += [
        {"seccion": i, "metodo": "Solo CARD (rgi)", "tiempo_min":_
    ↪m_card["tiempo_s"] / 60,
        "ram_pico_MB": m_card["ram_pico_MB"], "nucleo_horas":_
    ↪m_card["nucleo_segundos"] / 3600},
        {"seccion": i, "metodo": "Solo AMRFinderPlus", "tiempo_min":_
    ↪m_amr["tiempo_s"] / 60,
        "ram_pico_MB": m_amr["ram_pico_MB"], "nucleo_horas":_
    ↪m_amr["nucleo_segundos"] / 3600},
        {"seccion": i, "metodo": "Mixto (CARD + AMRFinderPlus)",
        "tiempo_min": (m_card["tiempo_s"] + m_amr["tiempo_s"]) / 60,
        "ram_pico_MB": max(m_card["ram_pico_MB"], m_amr["ram_pico_MB"]),
        "nucleo_horas": (m_card["nucleo_segundos"] + m_amr["nucleo_segundos"])_
    ↪/ 3600},
    ]

    cg, cf = cruce_gen[i], cruce_familia[i]
    ambos_g = int((cg["en_CARD"] & cg["en_AMRFinderPlus"]).sum())
    solo_card_g = int((cg["en_CARD"] & ~cg["en_AMRFinderPlus"]).sum())
    solo_amr_g = int((~cg["en_CARD"] & cg["en_AMRFinderPlus"]).sum())
    ambos_f = int((cf["en_CARD"] & cf["en_AMRFinderPlus"]).sum())
    solo_card_f = int((cf["en_CARD"] & ~cf["en_AMRFinderPlus"]).sum())
    solo_amr_f = int((~cf["en_CARD"] & cf["en_AMRFinderPlus"]).sum())

    filas_riq += [
        {"seccion": i, "metodo": "Solo CARD (rgi)", "genes": ambos_g +_
    ↪solo_card_g,
        "familias_clases": ambos_f + solo_card_f},
```

```

        {"seccion": i, "metodo": "Solo AMRFinderPlus", "genes": ambos_g +
↪solo_amr_g,
        "familias_clases": ambos_f + solo_amr_f},
        {"seccion": i, "metodo": "Mixto (CARD + AMRFinderPlus)", "genes":
↪len(cg), "familias_clases": len(cf)},
    ]

comparacion_eficiencia = pd.DataFrame(filas_efic)
comparacion_riqueza = pd.DataFrame(filas_riq)

print("Eficiencia por metodo y seccion:")
display(comparacion_eficiencia)

print("\nRiqueza de resultados por metodo y seccion (genes/familias distintos
↪detectados):")
display(comparacion_riqueza)

comparacion = comparacion_eficiencia.merge(comparacion_riqueza, on=["seccion",
↪"metodo"])
comparacion["genes_por_minuto"] = comparacion["genes"] /
↪comparacion["tiempo_min"]
print("\nRelacion costo/beneficio (genes distintos detectados por minuto de
↪computo):")
comparacion[["seccion", "metodo", "tiempo_min", "ram_pico_MB", "genes",
↪"familias_clases", "genes_por_minuto"]]

```

Eficiencia por metodo y seccion:

	seccion	metodo	tiempo_min	ram_pico_MB	\
0	1	Solo CARD (rgi)	0.249500	384.296875	
1	1	Solo AMRFinderPlus	2.408333	487.527344	
2	1	Mixto (CARD + AMRFinderPlus)	2.657833	487.527344	
3	2	Solo CARD (rgi)	0.160833	309.066406	
4	2	Solo AMRFinderPlus	1.996333	462.078125	
5	2	Mixto (CARD + AMRFinderPlus)	2.157167	462.078125	

	nucleo_horas
0	0.013348
1	0.430690
2	0.444039
3	0.009864
4	0.360338
5	0.370203

Riqueza de resultados por metodo y seccion (genes/familias distintos detectados):

seccion	metodo	genes	familias_clases
---------	--------	-------	-----------------

0	1	Solo CARD (rgi)	23	15
1	1	Solo AMRFinderPlus	23	9
2	1	Mixto (CARD + AMRFinderPlus)	35	18
3	2	Solo CARD (rgi)	17	11
4	2	Solo AMRFinderPlus	22	9
5	2	Mixto (CARD + AMRFinderPlus)	29	14

Relacion costo/beneficio (genes distintos detectados por minuto de computo):

```
[40]: seccion          metodo tiempo_min ram_pico_MB genes \
0      1              Solo CARD (rgi)  0.249500  384.296875  23
1      1              Solo AMRFinderPlus  2.408333  487.527344  23
2      1 Mixto (CARD + AMRFinderPlus)  2.657833  487.527344  35
3      2              Solo CARD (rgi)  0.160833  309.066406  17
4      2              Solo AMRFinderPlus  1.996333  462.078125  22
5      2 Mixto (CARD + AMRFinderPlus)  2.157167  462.078125  29
```

	familias_clases	genes_por_minuto
0	15	92.184369
1	9	9.550173
2	18	13.168621
3	11	105.699482
4	9	11.020204
5	14	13.443560

1.1.12 Resumen de la corrida individual, medido de punta a punta

```
[41]: medicion_individual = pd.DataFrame(mediciones)
medicion_individual["tiempo_min"] = medicion_individual["tiempo_s"] / 60
medicion_individual["nucleo_horas"] = medicion_individual["nucleo_segundos"] / 3600
medicion_individual["disco_MB"] = medicion_individual["disco_bytes"] / 1e6

nucleo_horas_computo = medicion_individual.loc[
    medicion_individual["paso"] != "descarga ENA", "nucleo_horas"
].sum()

print(f"Muestra: {MUESTRA_REF}")
print(f"Tiempo total (incluida la descarga): {medicion_individual['tiempo_s'].sum()/60:.1f} min")
print(f"Tiempo de computo local (sin la descarga): "
      f"{medicion_individual.loc[medicion_individual['paso'] != 'descarga ENA', 'tiempo_s'].sum()/60:.1f} min")
print(f"Nucleo-horas de computo real (sin la descarga -- la metrica que factura un HPC): "
      f"{nucleo_horas_computo:.3f}")
```

```
print(f"Disco total ocupado (suma de lo que deja cada paso, sin contar lo que
↳limpia el pipeline despues): "
      f"{medicion_individual['disco_MB'].sum()/1000:.2f}GB")

medicion_individual[["paso", "tiempo_min", "ram_pico_MB", "nucleo_horas",
↳"disco_MB", "nota"]]
```

Muestra: ERR1135179

Tiempo total (incluida la descarga): 38.4 min

Tiempo de computo local (sin la descarga): 30.4 min

Nucleo-horas de computo real (sin la descarga -- la metrica que factura un HPC):
4.391

Disco total ocupado (suma de lo que deja cada paso, sin contar lo que limpia el
pipeline despues): 13.37GB

```
[41]:
```

	paso	tiempo_min	ram_pico_MB	\
0	descarga ENA	7.978333	35.804688	
1	QC (fastp)	1.295333	1239.726562	
2	submuestreo (seqtk, aleatorio)	1.580000	2.371094	
3	ensamblado (MEGAHIT) secc.1	7.532500	841.523438	
4	ensamblado (MEGAHIT) secc.2	6.779500	795.773438	
5	prediccion de genes (prodigal) secc.1	1.511833	41.953125	
6	prediccion de genes (prodigal) secc.2	1.435667	42.085938	
7	descarga CARD DB	0.032000	10.593750	
8	carga/indexado CARD DB (rgi load)	0.145000	384.292969	
9	descarga AMRFinderPlus DB	0.629000	63.492188	
10	deteccion de ARG (rgi) secc.1	0.249500	384.296875	
11	deteccion de ARG (rgi) secc.2	0.160833	309.066406	
12	deteccion de ARG (AMRFinderPlus) secc.1	2.408333	487.527344	
13	deteccion de ARG (AMRFinderPlus) secc.2	1.996333	462.078125	
14	deteccion de ARG (rgi) secc.1	0.210333	310.902344	
15	deteccion de ARG (rgi) secc.2	0.151667	309.125000	
16	deteccion de ARG (AMRFinderPlus) secc.1	2.319000	476.062500	
17	deteccion de ARG (AMRFinderPlus) secc.2	1.986333	460.398438	

	nucleo_horas	disco_MB	\
0	0.009308	4404.958009	
1	0.211355	4538.926759	
2	0.026198	2843.236428	
3	1.316932	607.875811	
4	1.177373	549.076042	
5	0.024945	69.375346	
6	0.023689	60.731787	
7	0.000016	4.366637	
8	0.002429	38.086878	
9	0.001992	252.180449	
10	0.013348	0.635740	

11	0.009864	0.528529
12	0.430690	0.006651
13	0.360338	0.006402
14	0.010762	0.635740
15	0.009884	0.528529
16	0.415101	0.006651
17	0.356547	0.006402

	nota
0	velocidad real medida: 73.62 Mbps (aria2c, 12 ...
1	28036112 pares, 5.24Gbp
2	pool aleatorio de 5,608,125 pares (20.0% de 28...
3	NaN
4	NaN
5	55034 genes candidatos
6	48738 genes candidatos
7	velocidad real medida: 18.19 Mbps (wget, 1 con...
8	extraccion (tar) + indexado local (rgi load --...
9	velocidad real medida: 53.46 Mbps (amrfinder_u...
10	29 genes de resistencia
11	21 genes de resistencia
12	23 genes de resistencia
13	22 genes de resistencia
14	29 genes de resistencia
15	21 genes de resistencia
16	23 genes de resistencia
17	22 genes de resistencia

1.2 Cuánto varía el tamaño real de cada muestra

No descarga ninguna secuencia: consulta a la misma API de ENA el campo `fastq_bytes`, el tamaño comprimido exacto que ENA reporta tener listo para servir, para cada una de las 89 muestras seleccionadas (`muestreo_PAIRED_89.xlsx`, subconjunto fijo de las 295 corridas PAIRED profundas disponibles).

```
[42]: def bytes_fastq_ena(run_accession):
    url = (
        "https://www.ebi.ac.uk/ena/portal/api/filereport"
        f"?
↪accession={run_accession}&result=read_run&fields=fastq_bytes&format=tsv"
    )
    def _get():
        with urllib.request.urlopen(url, timeout=20) as r:
            return r.read().decode()
    texto = reintentar(_get, descripcion=f"consulta ENA bytes_
↪({run_accession})")
    lineas = texto.strip().split("\n")
```

```

    if len(lineas) < 2 or not lineas[1].split("\t")[-1]:
        return None
    valores = lineas[1].split("\t")[-1].split(",")
    return sum(int(v) for v in valores if v)

tam_por_muestra = {}
sin_datos = []
for run in RUNS:
    b = bytes_fastq_ena(run)
    if b is None:
        sin_datos.append(run)
    else:
        tam_por_muestra[run] = b

tam_muestras_gb = pd.Series(tam_por_muestra, name="GB") / 1e9
TOTAL_DESCARGA_GB = tam_muestras_gb.sum()

print(f"Consultadas {len(RUNS)} muestras; sin dato de tamaño: {sin_datos or 'ninguna'}")
print(f"Tamaño por muestra: min {tam_muestras_gb.min():.2f}GB, media {tam_muestras_gb.mean():.2f}GB, "
      f"max {tam_muestras_gb.max():.2f}GB, desviacion estandar {tam_muestras_gb.std():.2f}GB")
print(f"Descarga total del lote de {len(tam_muestras_gb)} muestras: {TOTAL_DESCARGA_GB:.1f} GB comprimidos")

```

```

Fallo en consulta ENA bytes (ERR1135328) (<urlopen error timed out>);
reintentando en 5s [1/5]...
Fallo en consulta ENA bytes (ERR1135329) (<urlopen error timed out>);
reintentando en 5s [1/5]...
Fallo en consulta ENA bytes (ERR1135425) (<urlopen error timed out>);
reintentando en 5s [1/5]...
Fallo en consulta ENA bytes (ERR1135393) (<urlopen error timed out>);
reintentando en 5s [1/5]...
Fallo en consulta ENA bytes (ERR1135403) (<urlopen error timed out>);
reintentando en 5s [1/5]...
Consultadas 89 muestras; sin dato de tamaño: ninguna
Tamaño por muestra: min 1.80GB, media 5.08GB, max 8.10GB, desviacion estandar 1.55GB
Descarga total del lote de 89 muestras: 451.9 GB comprimidos

```

1.3 Rango de cómputo

Esta sección proyecta las mediciones anteriores a núcleo-horas (núcleo-segundos = tiempo de pared $\times$ %CPU real que reportó `/usr/bin/time -v`, es decir consumo real de núcleos) para el rango real de tamaños reportado.

La descarga se deja por fuera de las núcleo-horas porque es E/S de red y casi no ocupa núcleo de CPU. Eso no significa que se haga fuera del HPC: los FASTQ se descargan en el HPC, y su costo se cuenta como tiempo de pared y disco en la estimación de la asignación.

Los demás pasos no escalan todos de la misma forma con el tamaño descargado:

- QC (fastp) y submuestreo (seqtk, aleatorio) procesan/recorren el archivo completo antes de aislar el 20% (10%+10%), así que su costo sí crece con cuánto pese la muestra original, se proyectan linealmente por GB. A diferencia de la versión posicional anterior, el submuestreo ahora es una sola llamada por muestra (no una por sección): el pool aleatorio del 20% se extrae una vez y se parte en las N_SECCIONES secciones después, así que ya no hay que sumar el costo de las dos secciones entre sí.
- Ensamblado (MEGAHIT), predicción de genes (prodigal) y detección de ARG (CARD/RGI y AMRFinderPlus, las dos) corren siempre sobre esas submuestras de tamaño fijo (10% cada una), nunca sobre la muestra completa. Su costo no depende de cuánto pesen los FASTQ originales (sólo de la diversidad de la comunidad microbiana, que no es función del tamaño en GB). Como ahora corren una vez por sección (N_SECCIONES veces por muestra), el valor medido por muestra ya es la suma de ambas secciones. Se reportan constantes frente al tamaño de la muestra, pero al doble de lo que costaría una sola submuestra.

```
[43]: CATEGORIA_ESCALAN = {
    "QC (fastp)": ["QC (fastp)"],
    "submuestreo (seqtk, aleatorio)": ["submuestreo (seqtk, aleatorio)"],
}
CATEGORIA_FIJOS = {
    "ensamblado (MEGAHIT)": [f"ensamblado (MEGAHIT) secc.{i}" for i in range(1, N_SECCIONES + 1)],
    "prediccion de genes (prodigal)": [f"prediccion de genes (prodigal) secc.{i}" for i in range(1, N_SECCIONES + 1)],
    "deteccion de ARG (CARD/RGI)": [f"deteccion de ARG (rgi) secc.{i}" for i in range(1, N_SECCIONES + 1)],
    "deteccion de ARG (AMRFinderPlus)": [f"deteccion de ARG (AMRFinderPlus) secc.{i}" for i in range(1, N_SECCIONES + 1)],
}

gb_referencia = bytes_descargados / 1e9
medido_paso = medicion_individual.set_index("paso")["nucleo_segundos"]
medido = pd.Series({categoria: medido_paso[pasos].sum()
                    for categoria, pasos in **CATEGORIA_ESCALAN,
                    **CATEGORIA_FIJOS}.items()})

filas_rango = []
for etiqueta, gb in [("minimo", tam_muestras_gb.min()), ("promedio", tam_muestras_gb.mean()),
                    ("maximo", tam_muestras_gb.max())]:
    for paso in CATEGORIA_ESCALAN:
        filas_rango.append({"paso": paso, "escenario": etiqueta,
```

```

        "nucleo_horas_estimadas": medido[paso] /
gb_referencia * gb / 3600})
    for paso in CATEGORIA_FIJOS:
        filas_rango.append({"paso": paso, "escenario": etiqueta,
            "nucleo_horas_estimadas": medido[paso] / 3600})

rango_individual = pd.DataFrame(filas_rango)
rango_individual_pivot = rango_individual.pivot(index="paso",
    columns="escenario", values="nucleo_horas_estimadas")
orden_pasos = list(CATEGORIA_ESCALAN) + list(CATEGORIA_FIJOS)
rango_individual_pivot = rango_individual_pivot.loc[orden_pasos, ["minimo",
    "promedio", "maximo"]].round(4)
nucleo_horas_por_muestra = rango_individual_pivot.sum()

print(f"Muestra de referencia ({MUESTRA_REF}): {gb_referencia:.2f}GB, "
    f"{medido.sum()/3600:.3f} nucleo-horas medidas ({N_SECCIONES} secciones)")
print(f"Rango real de tamaños en el lote: {tam_muestras_gb.min():.2f}GB -
    {tam_muestras_gb.max():.2f}GB")
print(f"Nucleo-horas de computo por muestra (todos los pasos, sin la descarga):
    ")
    f"minimo {nucleo_horas_por_muestra['minimo']:.3f}, promedio
    {nucleo_horas_por_muestra['promedio']:.3f}, "
    f"maximo {nucleo_horas_por_muestra['maximo']:.3f}\n")
print("Nucleo-horas estimadas por paso ('escalan': con el tamaño descargado;
    'fijo': constante porque "
    f"corren sobre las submuestras, no sobre la muestra completa -- ya suman
    las {N_SECCIONES} secciones):")
rango_individual_pivot

```

Muestra de referencia (ERR1135179): 4.40GB, 4.387 nucleo-horas medidas (2 secciones)

Rango real de tamaños en el lote: 1.80GB - 8.10GB

Nucleo-horas de computo por muestra (todos los pasos, sin la descarga): minimo 4.247, promedio 4.423, maximo 4.586

Nucleo-horas estimadas por paso ('escalan': con el tamaño descargado; 'fijo': constante porque corren sobre las submuestras, no sobre la muestra completa -- ya suman las 2 secciones):

[43]:	escenario	minimo	promedio	maximo
	paso			
	QC (fastp)	0.0865	0.2436	0.3886
	submuestreo (seqtk, aleatorio)	0.0107	0.0302	0.0482
	ensamblado (MEGAHIT)	2.4943	2.4943	2.4943
	prediccion de genes (prodigal)	0.0486	0.0486	0.0486
	deteccion de ARG (CARD/RGI)	0.0439	0.0439	0.0439
	deteccion de ARG (AMRFinderPlus)	1.5627	1.5627	1.5627

1.4 Volumen y tiempo real

Se representa por medio de una fórmula ($\text{tiempo} = \text{datos} / \text{velocidad}$), ya que depende de la velocidad de la red. Se proyecta dos veces: con la velocidad real que se midió en la sección 1 ($\text{velocidad_medida_mbps}$, red doméstica, solo de referencia) y con la velocidad real informada del enlace de descarga del HPC ($\text{VELOCIDAD_HPC_MBPS} = 1 \text{ Gbps}$), esta última es la que se va a utilizar para la solicitud.

Para la base *nt* (más abajo) se usa una velocidad distinta, $\text{velocidad_bases_mbps}$, el promedio de lo medido al descargar CARD y AMRFinderPlus en vez de $\text{velocidad_medida_mbps}$: los FASTQ se miden con *aria2c* (multi-conexión), pero *nt* se descarga con *update_blastdb.pl*, de una sola conexión como CARD/AMRFinderPlus, así que esa es la comparación representativa.

```
[44]: def horas_de_descarga.gb, mbps):
    """gb: gigabytes a transferir. mbps: megabits por segundo efectivos (no
    megabytes)."""
    megabits = gb * 8 * 1000
    return (megabits / mbps) / 3600

TU_VELOCIDAD_MBPS = None
VELOCIDAD_HPC_MBPS = 1000

velocidad_mbps = TU_VELOCIDAD_MBPS or velocidad_medida_mbps
fuente = "medida en la sección 1 (este entorno, no representativa de una
conexión normal)" \
    if TU_VELOCIDAD_MBPS is None else "definida manualmente arriba"

print(f"Usando {velocidad_mbps:.2f} Mbps ({fuente})\n")

print("Descarga de una muestra individual, según su tamaño real (sección 2):")
for etiqueta, gb in [("minimo", tam_muestras_gb.min()), ("promedio",
tam_muestras_gb.mean()),
("maximo", tam_muestras_gb.max())]:
    h = horas_de_descarga.gb, velocidad_mbps)
    print(f" {etiqueta} ({gb:.2f}GB): {h*60:.0f} min" if h < 1 else f"
{etiqueta} ({gb:.2f}GB): {h:.1f} horas")

h_total = horas_de_descarga(TOTAL_DESCARGA_GB, velocidad_mbps)
print(f"\nDescarga del lote completo ({TOTAL_DESCARGA_GB:.1f}GB, sección 2):
{h_total:.1f} horas")

NT_DB_TAMANO_GB_ESTIMADO = 400

velocidad_bases_mbps = (velocidad_card_mbps + velocidad_amr_mbps) / 2
h_nt = horas_de_descarga(NT_DB_TAMANO_GB_ESTIMADO, velocidad_bases_mbps)
print(f"\nDescarga UNICA de la base nt para BLAST local (04_pipeline_organizado.
ipynb, no se repite ")
```

```

        f"por muestra): ~{NT_DB_TAMANO_GB_ESTIMADO}GB estimados (sin verificar --\n
        ↪ver nota arriba) "
        f"-> ~{h_nt:.1f} horas a {velocidad_bases_mbps:.2f} Mbps (promedio CARD/\n
        ↪AMRFinderPlus medidos en "
        f"la seccion 1.5).")

print(f"\nA la velocidad del HPC ({VELOCIDAD_HPC_MBPS:.0f} Mbps =\n
    ↪{VELOCIDAD_HPC_MBPS/1000:.0f} Gbps, "
        f"informada, no medida en este entorno):")
for etiqueta, gb in [("minimo", tam_muestras_gb.min()), ("promedio",\n
    ↪tam_muestras_gb.mean()),
                    ("maximo", tam_muestras_gb.max())]:
    h = horas_de_descarga(gb, VELOCIDAD_HPC_MBPS)
    print(f"    {etiqueta} ({gb:.2f}GB): {h*60:.1f} min")

h_total_hpc = horas_de_descarga(TOTAL_DESCARGA_GB, VELOCIDAD_HPC_MBPS)
h_nt_hpc = horas_de_descarga(NT_DB_TAMANO_GB_ESTIMADO, VELOCIDAD_HPC_MBPS)
print(f"    Lote completo ({TOTAL_DESCARGA_GB:.1f}GB): {h_total_hpc*60:.1f} min")
print(f"    Base nt (~{NT_DB_TAMANO_GB_ESTIMADO}GB estimados, sin verificar):\n
    ↪{h_nt_hpc:.2f} horas")

```

Usando 73.62 Mbps (medida en la sección 1 (este entorno, no representativa de una conexión normal))

Descarga de una muestra individual, según su tamaño real (sección 2):

```

    minimo (1.80GB): 3 min
    promedio (5.08GB): 9 min
    maximo (8.10GB): 15 min

```

Descarga del lote completo (451.9GB, sección 2): ~13.6 horas

Descarga UNICA de la base nt para BLAST local (04_pipeline_organizado.ipynb, no se repite por muestra): ~400GB estimados (sin verificar -- ver nota arriba) -> ~24.8 horas a 35.83 Mbps (promedio CARD/AMRFinderPlus medidos en la seccion 1.5).

A la velocidad del HPC (1000 Mbps = 1 Gbps, informada, no medida en este entorno):

```

    minimo (1.80GB): 0.2 min
    promedio (5.08GB): 0.7 min
    maximo (8.10GB): 1.1 min
    Lote completo (451.9GB): 60.3 min
    Base nt (~400GB estimados, sin verificar): 0.89 horas

```

1.5 Estimación de la asignación a solicitar en el HPC de Biocómputo (cola `cpu.cecc`)

La descarga de los FASTQ y de las bases compartidas también se hace en el HPC, por la velocidad de su enlace y porque el lote completo no cabe en un equipo personal; entra en el walltime y en la cuota de disco, no en las núcleo-horas. Basado en los datos reportados en la Wiki del CECC, se estiman los siguientes recursos:

Parámetro	Valor de la cola
Tipo de usuario	Miembro del CECC
Tiempo máximo por trabajo (wall clock)	hasta 8 días
CPU/nodo	8 a 16
GPU/nodo	0
RAM/nodo	24GB a 96GB
Cuota de almacenamiento asignada	0TB (sin almacenamiento persistente por defecto)
Cuota para procesamiento (scratch)	máx. 2TB
Prioridad de recursos	Baja

```
[45]: CPUS_MIN, CPUS_MAX = 8, 16
RAM_MIN_GB, RAM_MAX_GB = 24, 96
WALLTIME_MAX_DIAS = 8
STORAGE_PROC_MAX_TB = 2
N_MUESTRAS = len(tam_muestras_gb)

CPUS_SOLICITADOS = THREADS

nucleo_s_escalable_lote = sum(medido[p] / gb_referencia * TOTAL_DESCARGA_GB for
    ↪p in CATEGORIA_ESCALAN)
nucleo_s_fijo_lote = sum(medido[p] for p in CATEGORIA_FIJOS) * N_MUESTRAS
nucleo_horas_lote = (nucleo_s_escalable_lote + nucleo_s_fijo_lote) / 3600

MARGEN_SEGURIDAD = 1.5
nucleo_horas_lote_con_margen = nucleo_horas_lote * MARGEN_SEGURIDAD

nucleo_horas_carga_card = medido_paso["carga/indexado CARD DB (rgi load)"] /
    ↪3600
nucleo_horas_lote_con_margen += nucleo_horas_carga_card

print(f"Nucleo-horas por muestra (seccion 3): minimo
    ↪{nucleo_horas_por_muestra['minimo']:.3f}, "
      f"promedio {nucleo_horas_por_muestra['promedio']:.3f}, maximo
    ↪{nucleo_horas_por_muestra['maximo']:.3f}")
print(f"Nucleo-horas para el lote completo de {N_MUESTRAS} muestras "
      f"({TOTAL_DESCARGA_GB:.1f}GB reales, seccion 2): {nucleo_horas_lote:.2f}")
print(f"Con margen de seguridad ({MARGEN_SEGURIDAD}x) + indexado de CARD (una
    ↪sola vez, seccion 1.5, "
```

```

        f"{nucleo_horas_carga_card:.4f} nucleo-horas):\n
        ↪{nucleo_horas_lote_con_margen:.2f} nucleo-horas\n")

ram_pico_medido_gb = medicion_individual["ram_pico_MB"].max() / 1024
MARGEN_RAM = 4
ram_recomendada_gb = max(RAM_MIN_GB, ram_pico_medido_gb * MARGEN_RAM)

print(f"RAM pico medida (peor paso, muestra de referencia): {ram_pico_medido_gb:
    ↪.2f}GB")
print(f"RAM recomendada a pedir por trabajo (margen {MARGEN_RAM}x, rango de la
    ↪cola {RAM_MIN_GB}-{RAM_MAX_GB}GB): "
        f"{ram_recomendada_gb:.1f}GB")
if ram_recomendada_gb > RAM_MAX_GB:
    print(f"    Aviso: supera el maximo de la cola ({RAM_MAX_GB}GB); pedir el
    ↪maximo disponible y monitorear.")

walltime_max_muestra_min = nucleo_horas_por_muestra["maximo"] /\n
    ↪CPUS_SOLICITADOS * 60
walltime_lote_horas = nucleo_horas_lote_con_margen / CPUS_SOLICITADOS

print(f"\nWalltime estimado para la muestra mas pesada del lote, con
    ↪{CPUS_SOLICITADOS} CPUs: "
        f"{walltime_max_muestra_min:.1f} min (limite de la cola:
    ↪{WALLTIME_MAX_DIAS} dias)")
print(f"Walltime si el lote completo corriera como un solo trabajo secuencial
    ↪con {CPUS_SOLICITADOS} CPUs: "
        f"{walltime_lote_horas:.2f} horas -- muy por debajo del limite; aun asi,
    ↪la recomendacion es un "
        f"trabajo por muestra (ver mas abajo), no uno monolitico.")

walltime_descarga_max_min = horas_de_descarga(tam_muestras_gb.max(),\n
    ↪VELOCIDAD_HPC_MBPS) * 60
print(f"\nSi la descarga corre dentro del mismo trabajo:
    ↪+{walltime_descarga_max_min:.1f} min para la "
        f"muestra mas pesada, a {VELOCIDAD_HPC_MBPS:.0f} Mbps
    ↪({VELOCIDAD_HPC_MBPS/1000:.0f} Gbps, "
        f"velocidad informada del HPC, seccion 4) -- sumar al walltime de arriba;
    ↪verificar primero si "
        f"el nodo de computo tiene salida a internet (ver 5.1) antes de asumir
    ↪que puede descargar algo.")

tiempo_prep_bases_s = (medido_paso["descarga CARD DB"] + medido_paso["carga/
    ↪indexado CARD DB (rgi load)"]
        + medido_paso["descarga AMRFinderPlus DB"])
print(f"\nPreparacion de bases compartidas (CARD + AMRFinderPlus, seccion 1.5,
    ↪una sola vez para todo ")

```

```

        f"el lote): {tiempo_prep_bases_s/60:.1f} min de pared medidos (descarga +
↳indexado); si corre "
        f"dentro del mismo trabajo, sumar tambien al walltime a pedir.")

CATEGORIA_ESCALAN_DISCO = {"QC (fastp)": ["QC (fastp)"]}
CATEGORIA_FIJOS_DISCO = {
    "submuestreo (seqtk, aleatorio)": ["submuestreo (seqtk, aleatorio)"],
    "ensamblado (MEGAHIT)": [f"ensamblado (MEGAHIT) secc.{i}" for i in range(1,
↳N_SECCIONES + 1)],
    "prediccion de genes (prodigal)": [f"prediccion de genes (prodigal) secc.
↳{i}" for i in range(1, N_SECCIONES + 1)],
    "deteccion de ARG (CARD/RGI)": [f"deteccion de ARG (rgi) secc.{i}" for i in
↳range(1, N_SECCIONES + 1)],
    "deteccion de ARG (AMRFinderPlus)": [f"deteccion de ARG (AMRFinderPlus)
↳secc.{i}" for i in range(1, N_SECCIONES + 1)],
}
disco_medido_paso = medicion_individual.set_index("paso")["disco_bytes"]
disco_medido = pd.Series({categoria: disco_medido_paso[pasos].sum()
    for categoria, pasos in **CATEGORIA_ESCALAN_DISCO,
↳**CATEGORIA_FIJOS_DISCO}.items()))

disco_escalable_lote = sum(disco_medido[p] / gb_referencia * TOTAL_DESCARGA_GB
↳for p in CATEGORIA_ESCALAN_DISCO)
disco_fijo_lote = sum(disco_medido[p] for p in CATEGORIA_FIJOS_DISCO) *
↳N_MUESTRAS
disco_lote_gb = (disco_escalable_lote + disco_fijo_lote) / 1e9
disco_lote_con_raw_gb = disco_lote_gb + TOTAL_DESCARGA_GB

print(f"\nDisco de resultados/intermedios proyectado para el lote completo:
↳{disco_lote_gb:.1f}GB")
print(f"Disco total si los FASTQ crudos conviven con los intermedios:
↳{disco_lote_con_raw_gb:.1f}GB de "
    f"{STORAGE_PROC_MAX_TB * 1000:.0f}GB de cuota de procesamiento "
    f"({disco_lote_con_raw_gb / (STORAGE_PROC_MAX_TB * 1000) * 100:.1f}% de
↳la cuota)")

CARD_DB_GB = tamano("localDB/card.json") / 1e9
AMRFINDERPLUS_DB_GB = tamano(AMRFINDERPLUS_DB_DIR) / 1e9
TAXDB_TAMANO_GB_ESTIMADO = 1

bases_compartidas_gb = CARD_DB_GB + AMRFINDERPLUS_DB_GB +
↳NT_DB_TAMANO_GB_ESTIMADO + TAXDB_TAMANO_GB_ESTIMADO
disco_lote_con_nt_gb = disco_lote_con_raw_gb + bases_compartidas_gb
pct_con_nt = disco_lote_con_nt_gb / (STORAGE_PROC_MAX_TB * 1000) * 100

print(f"\nBases compartidas (una sola vez, no por muestra): ")

```

```

f"CARD {CARD_DB_GB*1000:.1f}MB (medida), AMRFinderPlus_
↳{AMRFINDERPLUS_DB_GB*1000:.1f}MB (medida), "
f"nt ~{NT_DB_TAMANO_GB_ESTIMADO}GB (estimado NCBI, sin verificar), "
f"taxdb ~{TAXDB_TAMANO_GB_ESTIMADO}GB (estimado NCBI, sin verificar)"
print(f"Con las bases compartidas: {disco_lote_con_nt_gb:.1f}GB de_
↳{STORAGE_PROC_MAX_TB * 1000:.0f}GB "
f"de cuota ({pct_con_nt:.1f}% de la cuota)")
if disco_lote_con_nt_gb > STORAGE_PROC_MAX_TB * 1000:
    print(" Aviso: supera la cuota de scratch de la cola -- verificar la cifra_
↳real de nt/taxdb antes "
f"de pedir la cuota; puede hacer falta pedir cuota adicional, correr_
↳el lote de muestras en "
f"tandas que liberen espacio entre si, o usar una base BLAST mas chica_
↳que nt completa.")

```

Nucleo-horas por muestra (seccion 3): minimo 4.247, promedio 4.423, maximo 4.586
Nucleo-horas para el lote completo de 89 muestras (451.9GB reales, seccion 2):
393.67
Con margen de seguridad (1.5x) + indexado de CARD (una sola vez, seccion 1.5,
0.0024 nucleo-horas): 590.51 nucleo-horas

RAM pico medida (peor paso, muestra de referencia): 1.21GB
RAM recomendada a pedir por trabajo (margen 4x, rango de la cola 24-96GB):
24.0GB

Walltime estimado para la muestra mas pesada del lote, con 12 CPUs: 22.9 min
(limite de la cola: 8 dias)
Walltime si el lote completo corriera como un solo trabajo secuencial con 12
CPUs: 49.21 horas -- muy por debajo del limite; aun asi, la recomendacion es un
trabajo por muestra (ver mas abajo), no uno monolitico.

Si la descarga corre dentro del mismo trabajo: +1.1 min para la muestra mas
pesada, a 1000 Mbps (1 Gbps, velocidad informada del HPC, seccion 4) -- sumar al
walltime de arriba; verificar primero si el nodo de computo tiene salida a
internet (ver 5.1) antes de asumir que puede descargar algo.

Preparacion de bases compartidas (CARD + AMRFinderPlus, seccion 1.5, una sola
vez para todo el lote): 0.3 min de pared medidos (descarga + indexado); si corre
dentro del mismo trabajo, sumar tambien al walltime a pedir.

Disco de resultados/intermedios proyectado para el lote completo: 833.5GB
Disco total si los FASTQ crudos conviven con los intermedios: 1285.4GB de 2000GB
de cuota de procesamiento (64.3% de la cuota)

Bases compartidas (una sola vez, no por muestra): CARD 38.1MB (medida),
AMRFinderPlus 252.2MB (medida), nt ~400GB (estimado NCBI, sin verificar), taxdb
~1GB (estimado NCBI, sin verificar)

Con las bases compartidas: 1686.7GB de 2000GB de cuota (84.3% de la cuota)

1.5.1 Software y otros requerimientos del trabajo

- Entorno: `environment.yml` (conda/mamba, entorno `amr-ml`) fija todas las herramientas del pipeline, se recrea igual en el HPC (`conda env create -f environment.yml`). Las versiones reales que corrieron en esta muestra de referencia se listan abajo.
- Bases de datos locales: `localDB/card.json` y `localDB/amrfinderplus/`.
- Acceso a internet: la descarga de ENA y la actualización de las bases (`amrfinder_update`, `update_blastdb.pl`) necesitan salida a internet.
- N° de trabajos: un trabajo por muestra, no uno monolítico. Se piden `N_MUESTRAS` trabajos (o un job array de ese tamaño), cada uno con los CPU/RAM/walltime estimados arriba.

```
[46]: herramientas = {
    "aria2c": ["aria2c", "--version"],
    "fastp": ["fastp", "--version"],
    "megahit": ["megahit", "--version"],
    "prodigal": ["prodigal", "-v"],
    "rgi": ["rgi", "main", "--version"],
    "amrfinder": ["amrfinder", "--version"],
    "blastn": ["blastn", "-version"],
}

print(f"python: {sys.version.split()[0]}")
for nombre, cmd in herramientas.items():
    if shutil.which(cmd[0]) is None:
        print(f"{nombre}: NO ENCONTRADO en este entorno")
        continue
    salida = subprocess.run(cmd, text=True, capture_output=True)
    linea = (salida.stdout or salida.stderr).strip().splitlines()
    print(f"{nombre}: {linea[0] if linea else '(sin salida de version)'}")

if shutil.which("rgi") is not None:
    card_ver = subprocess.run(["rgi", "database", "--version"], text=True,
    ↪capture_output=True)
    print(f"CARD data (rgi database --version): {card_ver.stdout.strip() or
    ↪card_ver.stderr.strip()}")

print(f"\nEntorno: conda env create -f environment.yml (entorno 'amr-ml')")
print(f"CARD local: {CARD_DB_GB*1000:.1f}MB en localDB/card.json (versionada en
    ↪git)")
print(f"AMRFinderPlus DB local: {AMRFINDERPLUS_DB_GB*1000:.1f}MB en
    ↪{AMRFINDERPLUS_DB_DIR} "
    f"(no versionada, se descarga con amrfinder_update -- medido en la
    ↪seccion 1.5)")
```

```
print(f"\nTrabajos a enviar: {N_MUESTRAS} (uno por muestra) con_
↳{CPUS_SOLICITADOS} CPUs, "
      f"{ram_recomendada_gb:.1f}GB RAM y hasta {WALLTIME_MAX_DIAS} dias de_
↳walltime cada uno")
```

```
python: 3.11.15
aria2c: aria2 version 1.37.0
fastp: fastp 1.3.6
megahit: MEGAHIT v1.2.9
prodigal: Prodigal V2.6.3: February, 2016
rgi: 6.0.8
amrfinder: 4.2.7
blastn: blastn: 2.16.0+
CARD data (rgi database --version): 4.0.1
```

```
Entorno: conda env create -f environment.yml (entorno 'amr-ml')
CARD local: 38.1MB en localDB/card.json (versionada en git)
AMRFinderPlus DB local: 252.2MB en localDB/amrfinderplus (no versionada, se
descarga con amrfinder_update -- medido en la seccion 1.5)
```

Trabajos a enviar: 89 (uno por muestra) con 12 CPUs, 24.0GB RAM y hasta 8 dias de walltime cada uno

1.6 Vinculación a especie y posición genómica de referencia (BLAST) – referencia

El código de la celda de abajo queda como referencia en markdown, no como celda ejecutable, por dos razones que ya no aplican a 04 porque ese notebook es HPC-only: Requiere la base nt completa de NCBI descargada localmente (cientos de GB) para `blastn`.

Lo único que sí se agrega en este notebook es el costo de esa base nt, disco y tiempo de descarga única porque ese costo sí hay que pedirlo en la cuota del HPC.

Referencia – código real y actualizado en 04_pipeline_organizado.ipynb (secciones “Base BLAST local” y 3.7). No ejecutable aquí.

```
BLASTDB_DIR = Path("localDB/blastdb")
```

```
NT_DB = BLASTDB_DIR / "nt"
```

```
TAXID_BACTERIA = "2" # taxid de NCBI para el dominio Bacteria
```

```
def ensure_blast_nt_db():
```

```
    # update_blastdb.pl --decompress --source ncbi nt (una sola vez, compartida -- cientos d
    # update_blastdb.pl --decompress taxdb (necesaria para -taxids)
    ...
```

```
def blast_local_taxonomia_y_posicion(contigs_recs, out_dir, tag):
```

```
    # Un solo blastn por muestra, no uno por contig: multi-FASTA con todos los contigs con ARG
```

```

# blastn -query {multi_fasta} -db {NT_DB} -task megablast -taxids {TAXID_BACTERIA}
#         -max_target_seqs 1 -max_hsps 1 -outfmt "6 qseqid sacc slen qstart qend sstart send
#         -num_threads {THREADS} -out {out_tsv}
# Devuelve taxon del mejor hit + coordenadas del alineamiento (contig y referencia).
...

def proyectar_posicion_en_referencia(row):
    # Interpola linealmente Start/Stop del ARG sobre el tramo que blastn alinea
    # (query_start-query_end -> sbjct_start-sbjct_end). Solo valida si el ARG cae dentro de es
    ...

def vincular_arg_a_especie(rgi, amrfinder, contigs, out_dir, tag):
    # Une los contigs con ARG de CARD y AMRFinderPlus, corre blast_local_taxonomia_y_posicion(
    # vez sobre esa union, y enriquece ambos dataframes con el mismo resultado.
    ...

```